Rake_Anthony_1_notebook_082025

August 6, 2025

ANALYSE DU STOCK ET DES VENTES DU SITE BOTTLENECK

1 OBJECTIF DE CE NOTEBOOK

Bienvenue dans l'outil plébiscité par les analystes de données Jupyter.

Il s'agit d'un outil permettant de mixer et d'alterner code, texte et graphiques.

Cet outil est formidable pour plusieurs raisons:

- Il permet de tester des lignes de codes au fur et à mesure de votre rédaction, de constater immédiatement le résultat d'une instruction, de la corriger si nécessaire.
- Il permet aussi de rédiger du texte pour expliquer l'approche suivie ou les résultats d'une analyse et de le mettre en forme grâce à du code html ou plus simple avec **Markdown**
- Il est possible d'ajouter des graphiques

Pour vous aider dans vos premiers pas à l'usage de Jupyter et de Python, nous avons rédigé ce notebook en vous indiquant les instructions à suivre.

Il vous suffit pour cela de saisir le code Python répondant à l'instruction donnée.

Vous verrez de temps à autre le code Python répondant à une instruction donnée mais cela est fait pour vous aider à comprendre la nature du travail qui vous est demandé.

Et gardez à l'esprit qu'il n'y a pas de solution unique pour résoudre un problème et qu'il y a autant de résolutions de problèmes que de développeurs ;)…

Etape 1 - Importation des librairies et chargement des fichiers

1.1 - Importation des librairies

[5]:
```
#Importation de la librairie Pandas
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
```

[6]:
```
#Importation de la librairie plotly express
import plotly.express as px
```

[7]:
```
#Trouver dans Google l'instruction permettant d'afficher toutes les colonnes␣
↪d'un dataframe
```

```
#Saisir dans Google les mots clés "display all columns dataframe Pandas" par␣
↪exemple.
#Dans les résultats de la recherche, privilégier les solutions provenant de␣
↪Stack Overflow ou Medium
```

1.2 - Chargements des fichiers

[9]:
```
#Importation du fichier web.xlsx
df_web = pd.read_excel("web.xlsx")
#Importation du fichier erp.xlsx
df_erp = pd.read_excel("erp.xlsx")
#Importation du fichier liaison.xlsx
df_liaison = pd.read_excel("liaison.xlsx")
```

```
C:\Users\t_rak\anaconda3\Lib\site-packages\openpyxl\worksheet\_read_only.py:85:
UserWarning: Unknown extension is not supported and will be removed
  for idx, row in parser.parse():
C:\Users\t_rak\anaconda3\Lib\site-packages\openpyxl\worksheet\_read_only.py:85:
UserWarning: Unknown extension is not supported and will be removed
  for idx, row in parser.parse():
C:\Users\t_rak\anaconda3\Lib\site-packages\openpyxl\worksheet\_read_only.py:85:
UserWarning: Unknown extension is not supported and will be removed
  for idx, row in parser.parse():
```

Etape 2 - Analyse exploratoire des fichiers

2.1 - Analyse exploratoire du fichier erp.xlsx

[12]:
```
#Afficher les dimensions du dataset
print("Le tableau comporte {} observation(s) ou article(s)".format(df_erp.
↪shape[0]))
print("Le tableau comporte {} colonne(s)".format(df_erp.shape[1]))
```

```
Le tableau comporte 825 observation(s) ou article(s)
Le tableau comporte 6 colonne(s)
```

[13]:
```
#Consulter le nombre de colonnes
#La nature des données dans chacune des colonnes
#Le nombre de valeurs présentes dans chacune des colonnes

# Nombre de colonnes
nb_colonnes = df_erp.shape[1]

# Nature des données dans chaque colonne
types_colonnes = df_erp.dtypes

# Nombre de valeurs non nulles dans chaque colonne
valeurs_non_nulles = df_erp.count()
```

```python
# Résumé global (optionnel pour tout voir en un coup d'œil)
resume = df_erp.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 825 entries, 0 to 824
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   product_id      825 non-null    int64
 1   onsale_web      825 non-null    int64
 2   price           825 non-null    float64
 3   stock_quantity  825 non-null    int64
 4   stock_status    825 non-null    object
 5   purchase_price  825 non-null    float64
dtypes: float64(2), int64(3), object(1)
memory usage: 38.8+ KB
```

[14]:
```python
#Afficher les 5 premières lignes de la table
df_erp.head(5)
```

[14]:

	product_id	onsale_web	price	stock_quantity	stock_status	purchase_price
0	3847	1	24.2	16	instock	12.88
1	3849	1	34.3	10	instock	17.54
2	3850	1	20.8	0	outofstock	10.64
3	4032	1	14.1	26	instock	6.92
4	4039	1	46.0	3	outofstock	23.77

[15]:
```python
#Vérifier si il y a des lignes en doublon dans la colonne product_id
doublons_erp = df_erp[df_erp.duplicated(subset='product_id', keep=False)]
doublons_erp
```

[15]:
```
Empty DataFrame
Columns: [product_id, onsale_web, price, stock_quantity, stock_status,
purchase_price]
Index: []
```

[16]:
```python
#Afficher les valeurs distinctes de la colonne stock_status
#À quelle(s) autre(s) colonne(s) sont-elles liées ? stock_quantity
valeurs_uniques = df_erp['stock_status'].unique()
valeurs_uniques
```

[16]:
```
array(['instock', 'outofstock'], dtype=object)
```

[17]:
```python
#Création d'une colonne "stock_status_2"
#La valeur de cette deuxième colonne sera fonction de la valeur dans la colonne␣
↪"stock_quantity"
#Si la valeur de la colonne "stock_quantity" est nulle, renseigner "outofstock"␣
↪sinon mettre "instock"
```

```
df_erp['stock_status_2'] = df_erp['stock_quantity'].apply(
    lambda x: 'outofstock' if pd.isna(x) or x == 0 else 'instock'
)
df_erp
```

[17]:

```
     product_id  onsale_web  price  stock_quantity stock_status  \
0          3847           1   24.2              16      instock
1          3849           1   34.3              10      instock
2          3850           1   20.8               0   outofstock
3          4032           1   14.1              26      instock
4          4039           1   46.0               3   outofstock
..          …          …    …              …           …
820        7203           0   45.0              30      instock
821        7204           0   45.0               9      instock
822        7247           1   54.8               6      instock
823        7329           0   26.5              14      instock
824        7338           1   16.3              40      instock

     purchase_price stock_status_2
0             12.88        instock
1             17.54        instock
2             10.64     outofstock
3              6.92        instock
4             23.77        instock
..              …            …
820           23.48        instock
821           24.18        instock
822           27.18        instock
823           13.42        instock
824            8.00        instock

[825 rows x 7 columns]
```

[18]:

```
#Vérifions que les 2 colonnes sont identiques:
#Les 2 colonnes sont strictement identiques si les valeurs de chaque ligne sont␣
↪strictement identiques 2 à 2
#La comparaison de 2 colonnes peut se réaliser simplement avec l'instruction␣
↪ci-dessous:
df_erp["stock_status"] == df_erp["stock_status_2"]

#Le résultat est l'affichage de True ou False pour chacune des lignes du dataset
#C'est un bon début, mais difficile à exploiter
```

[18]:

```
0       True
1       True
2       True
3       True
```

```
4      False
       …
820     True
821     True
822     True
823     True
824     True
Length: 825, dtype: bool
```

[19]:
```
#Mais il est possible de synthétiser ce résultat en effectuant la somme de␣
↪cette colonne:
#True vaut 1 et False 0
#Nous devrions obtenir la somme de 824 qui correspond au nombre de lignes dans␣
↪ce dataset
(df_erp["stock_status"] == df_erp["stock_status_2"]).sum()
```

[19]: 821

[20]:
```
#Si les colonnes ne sont absolument pas identiques ligne à ligne alors␣
↪identifier la ligne en écart
##Dans ce cas je vous donne ce lien pour apprendre à réaliser des filtres dans␣
↪Pandas:
##https://bitbucket.org/hrojas/learn-pandas/src/master/
##Lesson 3
ecarts = df_erp[df_erp["stock_status"] != df_erp["stock_status_2"]]
ecarts
```

[20]:
```
     product_id  onsale_web  price  stock_quantity stock_status  \
4          4039           1   46.0               3   outofstock
398        4885           1   18.7               0      instock
449        4973           0   10.0             -10   outofstock
573        5700           1   44.5              -1   outofstock

     purchase_price stock_status_2
4             23.77        instock
398            9.66     outofstock
449            4.96        instock
573           22.30        instock
```

[21]:
```
#Corriger la ou les données incohérentes
df_erp.loc[df_erp["stock_status"] != df_erp["stock_status_2"], "stock_status"]␣
↪= df_erp["stock_status_2"]
#Vérification en utilisant le même code que plus haut pour afficher les␣
↪problèmes
(df_erp["stock_status"] == df_erp["stock_status_2"]).sum()
```

[21]: 825

2.1.1 - Analyse exploratoire de chaque variable du fichier erp.xlsx

2.1.1.1 - Analyse de la variable PRIX

[24]:
```
df_erp['price'].describe()
```

[24]:
```
count    825.000000
mean      32.187697
std       26.712077
min      -20.000000
25%       14.500000
50%       24.300000
75%       42.000000
max      225.000000
Name: price, dtype: float64
```

[25]:
```
###############
## LES PRIX  ##
###############

#Vérification des prix: Y a t-il des prix non renseignés, négatifs ou nuls?
#Afficher le ou les prix non renseignés dans la colonne "price"
print(df_erp[df_erp['price'].isna()])
#print("Nombres d'articles avec un prix non renseigné: {}".format()) #Saisir␣
 ↪l'instruction manquante dans la fonction format
#Afficher le prix minimum de la colonne "price"
print("Le prix minimum est : {} euros".format(df_erp['price'].min()))
#Afficher le prix maximum de la colonne "price"
print(df_erp['price'].max())
#Afficher les prix inférieurs à 0 (qu'est-ce qu'il faut en faire ?)
print(df_erp[df_erp['price']<0])
```

```
Empty DataFrame
Columns: [product_id, onsale_web, price, stock_quantity, stock_status,
purchase_price, stock_status_2]
Index: []
Le prix minimum est : -20.0 euros
225.0
     product_id  onsale_web  price  stock_quantity stock_status  \
151        4233           0  -20.0               0   outofstock
469        5017           0   -8.0               0   outofstock
739        6594           0   -9.1              19      instock

     purchase_price stock_status_2
151           10.33     outofstock
469            4.34     outofstock
739            4.61        instock
```

[26]:
```
#que faire des prix négatifs ? Les remplacer par leurs valeurs absolues après␣
↪validation du client
df_erp['price']=df_erp['price'].abs()
# on applique directement la méthode abs() dans la colonne 'price' pour␣
↪transformer les valeurs négatives en positives
```

[27]:
```
df_erp[df_erp['price']<0]
```

[27]:
```
Empty DataFrame
Columns: [product_id, onsale_web, price, stock_quantity, stock_status,
purchase_price, stock_status_2]
Index: []
```

2.1.1.2 - Analyse de la variable STOCK

[29]:
```
#######################
### stock_quantity  ###
#######################

#Vérification de la colonne stock quantity
print(df_erp[df_erp['stock_quantity'].isna()])
#Afficher la quantité minimum de la colonne "stock_quantity"
print("La quantité minimum est : {} unités".format(df_erp['stock_quantity'].
↪min()))
#Afficher la quantité maximum de la colonne "stock_quantity"
print("La quantité ,maximum est : {} unités".format(df_erp['stock_quantity'].
↪max()))
#Afficher les stocks inférieurs à 0 (qu'est-ce qu'il faut en faire ?) Les␣
↪remplacer par leurs valeurs absolues
print(df_erp[df_erp['stock_quantity']<0])
```

```
Empty DataFrame
Columns: [product_id, onsale_web, price, stock_quantity, stock_status,
purchase_price, stock_status_2]
Index: []
La quantité minimum est : -10 unités
La quantité ,maximum est : 145 unités
     product_id  onsale_web  price  stock_quantity stock_status  \
449        4973           0   10.0             -10      instock
573        5700           1   44.5              -1      instock

     purchase_price stock_status_2
449            4.96        instock
573           22.30        instock
```

[30]:
```
#Remplacement des valeurs négatives de la colonne stock_quantity par leurs␣
↪valeurs absolues
df_erp['stock_quantity']=df_erp['stock_quantity'].abs()
```

```
print(df_erp[df_erp['stock_quantity']<0])
```

```
Empty DataFrame
Columns: [product_id, onsale_web, price, stock_quantity, stock_status,
purchase_price, stock_status_2]
Index: []
```

2.1.1.3 - Analyse de la variable ONSALE_WEB

[32]:
```
#Vérification de la colonne onsale_web et des valeurs qu'elle contient. Que␣
↪signifient-elles? 1=produit vendu en ligne et 0= non vendu en ligne
print("La valeur min est : {}".format(df_erp['onsale_web'].min()))
print("La valeur max est : {}".format(df_erp['onsale_web'].max()))
print(df_erp[df_erp['onsale_web']<0])
```

```
La valeur min est : 0
La valeur max est : 1
Empty DataFrame
Columns: [product_id, onsale_web, price, stock_quantity, stock_status,
purchase_price, stock_status_2]
Index: []
```

[33]:
```
#Vérification avec méthode UNIQUE
df_erp['onsale_web'].unique()
```

[33]:
```
array([1, 0], dtype=int64)
```

[34]:
```
#Vérification avec méthode nUNIQUE (indique le nb de valeurs)
df_erp['onsale_web'].nunique()
```

[34]:
```
2
```

[35]:
```
## exploration autres colonnes quand onsale_web = 0
df_non_vendus_en_ligne = df_erp[df_erp["onsale_web"] == 0]
df_non_vendus_en_ligne
```

[35]:
```
     product_id  onsale_web  price  stock_quantity stock_status  \
19         4055           0   86.1               0   outofstock
49         4090           0   73.0               0   outofstock
50         4092           0   47.0               0   outofstock
119        4195           0   14.1               0   outofstock
123        4200           0    5.8              33      instock
..          …          …     …               …           …
818        7200           0   31.0               6      instock
819        7201           0   31.0              18      instock
820        7203           0   45.0              30      instock
821        7204           0   45.0               9      instock
823        7329           0   26.5              14      instock
```

```
     purchase_price stock_status_2
19            37.88     outofstock
49            33.79     outofstock
50            25.25     outofstock
119            7.36     outofstock
123            3.12        instock
..              …            …
818           15.54        instock
819           16.02        instock
820           23.48        instock
821           24.18        instock
823           13.42        instock

[109 rows x 7 columns]
```

[36]:
```python
## exploration autres colonnes quand onsale_web = 1
df_vendus_en_ligne = df_erp[df_erp["onsale_web"] == 1]
df_vendus_en_ligne
```

[36]:
```
     product_id  onsale_web  price  stock_quantity stock_status  \
0          3847           1   24.2              16      instock
1          3849           1   34.3              10      instock
2          3850           1   20.8               0   outofstock
3          4032           1   14.1              26      instock
4          4039           1   46.0               3      instock
..          …         …     …              …           …
787        6930           1    8.4              28      instock
792        7023           1   27.5              23      instock
793        7025           1   69.0               8      instock
822        7247           1   54.8               6      instock
824        7338           1   16.3              40      instock

     purchase_price stock_status_2
0             12.88        instock
1             17.54        instock
2             10.64     outofstock
3              6.92        instock
4             23.77        instock
..              …            …
787            4.34        instock
792           14.21        instock
793           34.22        instock
822           27.18        instock
824            8.00        instock

[716 rows x 7 columns]
```

[37]:
```
# Que signifient-elles? -> signifie : est ce que le produit en question se vend␣
↪en ligne ou pas. 1=oui / 0=non
```

[38]:
```
#Quelles sont les colonnes à conserver selon vous?
df_vendus_en_ligne2 = df_vendus_en_ligne.drop(['onsale_web', 'stock_status_2'],␣
↪axis = 1)
df_vendus_en_ligne2
```

[38]:
```
     product_id  price  stock_quantity stock_status  purchase_price
0          3847   24.2              16      instock           12.88
1          3849   34.3              10      instock           17.54
2          3850   20.8               0   outofstock           10.64
3          4032   14.1              26      instock            6.92
4          4039   46.0               3      instock           23.77
..          …     …              …           …              …
787        6930    8.4              28      instock            4.34
792        7023   27.5              23      instock           14.21
793        7025   69.0               8      instock           34.22
822        7247   54.8               6      instock           27.18
824        7338   16.3              40      instock            8.00

[716 rows x 5 columns]
```

2.1.1.4 - Analyse de la variable prix d'achat

[40]:
```
######################
##   prix d'achat   ##
######################

#Vérification de la colonne purchase_price :
#Afficher le ou les prix non renseignés dans la colonne "purchase_price"
print(df_vendus_en_ligne2[df_vendus_en_ligne2['purchase_price'].isna()])
#Afficher le prix minimum de la colonne "purchase_price"
print("La prix d'achat min est : {} euros".
↪format(df_vendus_en_ligne2['purchase_price'].min()))
#Afficher le prix maximum de la colonne "purchase_price"
print("La prix d'achat max est : {} euros".
↪format(df_vendus_en_ligne2['purchase_price'].max()))
```

```
Empty DataFrame
Columns: [product_id, price, stock_quantity, stock_status, purchase_price]
Index: []
La prix d'achat min est : 2.74 euros
La prix d'achat max est : 137.81 euros
```

2.2 - Analyse exploratoire du fichier web.xlsx

[42]:
```
df_web['post_mime_type'].unique()
```

[42]: array(['image/jpeg', nan], dtype=object)

[43]:
```
##valeurs uniques df web
df_web.nunique()
```

[43]:
```
sku                      714
virtual                    1
downloadable               1
rating_count               1
average_rating             1
total_sales               29
tax_status                 1
tax_class                  0
post_author                2
post_date                716
post_date_gmt            716
post_content               0
product_type               7
post_title               713
post_excerpt             679
post_status                1
comment_status             1
ping_status                1
post_password              0
post_name                716
post_modified            589
post_modified_gmt        589
post_content_filtered      0
post_parent                1
guid                    1430
menu_order                 1
post_type                  2
post_mime_type             1
comment_count              1
dtype: int64
```

[44]:
```
df_web.columns
```

[44]:
```
Index(['sku', 'virtual', 'downloadable', 'rating_count', 'average_rating',
       'total_sales', 'tax_status', 'tax_class', 'post_author', 'post_date',
       'post_date_gmt', 'post_content', 'product_type', 'post_title',
       'post_excerpt', 'post_status', 'comment_status', 'ping_status',
       'post_password', 'post_name', 'post_modified', 'post_modified_gmt',
       'post_content_filtered', 'post_parent', 'guid', 'menu_order',
       'post_type', 'post_mime_type', 'comment_count'],
      dtype='object')
```

[45]:
```
#Dimension du dataset
#Nombre d'observations
print("Le tableau comporte {} observation(s) ou article(s)".format(df_web.
 ↪shape[0]))
#Nombre de caractéristiques
print("Le tableau comporte {} colonne(s)".format(df_web.shape[1]))
```

```
Le tableau comporte 1513 observation(s) ou article(s)
Le tableau comporte 29 colonne(s)
```

[46]:
```
#Consulter le nombre de colonnes
#La nature des données dans chacune des colonnes
#Le nombre de valeurs présentes dans chacune des colonnes
df_web.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1513 entries, 0 to 1512
Data columns (total 29 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   sku                    1428 non-null   object
 1   virtual                1513 non-null   int64
 2   downloadable           1513 non-null   int64
 3   rating_count           1513 non-null   int64
 4   average_rating         1430 non-null   float64
 5   total_sales            1430 non-null   float64
 6   tax_status             716 non-null    object
 7   tax_class              0 non-null      float64
 8   post_author            1430 non-null   float64
 9   post_date              1430 non-null   datetime64[ns]
 10  post_date_gmt          1430 non-null   datetime64[ns]
 11  post_content           0 non-null      float64
 12  product_type           1429 non-null   object
 13  post_title             1430 non-null   object
 14  post_excerpt           716 non-null    object
 15  post_status            1430 non-null   object
 16  comment_status         1430 non-null   object
 17  ping_status            1430 non-null   object
 18  post_password          0 non-null      float64
 19  post_name              1430 non-null   object
 20  post_modified          1430 non-null   datetime64[ns]
 21  post_modified_gmt      1430 non-null   datetime64[ns]
 22  post_content_filtered  0 non-null      float64
 23  post_parent            1430 non-null   float64
 24  guid                   1430 non-null   object
 25  menu_order             1430 non-null   float64
 26  post_type              1430 non-null   object
 27  post_mime_type         714 non-null    object
```

```
 28  comment_count          1430 non-null   float64
dtypes: datetime64[ns](4), float64(10), int64(3), object(12)
memory usage: 342.9+ KB
```

[47]:
```
#Selon vous, quelles sont les colonnes à conserver ?
df_web_clean = df_web.
 ↪drop(['post_title','post_excerpt','post_status','post_date','guid','virtual',␣
 ↪'downloadable', 'rating_count',␣
 ↪'average_rating','tax_class','post_content','post_password','post_content_filtered','post_a
 ↪'ping_status','post_modified_gmt','post_parent','menu_order',␣
 ↪'post_mime_type','comment_count'],axis=1)
df_web_clean
```

[47]:
```
        sku  total_sales tax_status product_type  \
0     11862          3.0        NaN          Vin
1     16057          5.0        NaN          Vin
2     14692          5.0    taxable          Vin
3     16295         14.0        NaN          Vin
4     15328          2.0    taxable          Vin
…     …           …         …           …
1508  16326          5.0    taxable          Vin
1509  15662         15.0    taxable          Vin
1510  15329          3.0        NaN          Vin
1511  14827          7.0        NaN          Vin
1512  16004          5.0        NaN          Vin

                                                 post_name       post_modified  \
0                            gilles-robin-hermitage-2012 2019-01-31 12:12:56
1            pelle-sancerre-rouge-la-croix-au-garde-2017 2020-07-07 10:05:02
2                  fonreaud-bordeaux-blanc-le-cygne-2016 2020-04-25 21:40:31
3     moulin-de-gassac-igp-pays-dherault-guilhem-ros… 2020-08-27 18:55:03
4                   agnes-levet-cote-rotie-maestria-2017 2020-07-25 15:45:02
…                                                   …                  …
1508       camin-larredya-jurancon-moelleux-capceu-2018 2020-08-27 09:24:46
1509  chermette-domaine-du-vissoux-beaujolais-griott… 2020-07-11 11:25:03
1510                agnes-levet-cote-rotie-peroline-2017 2020-07-28 09:45:02
1511  marc-colin-et-fils-chassagne-montrachet-blanc-… 2020-07-28 09:45:02
1512                     chateau-du-couvent-pomerol-2017 2020-08-25 18:35:02

       post_type
0     attachment
1     attachment
2        product
3     attachment
4        product
…           …
1508     product
```

```
1509     product
1510  attachment
1511  attachment
1512  attachment

[1513 rows x 7 columns]
```

[48]:
```python
#Si vous avez défini des colonnes à supprimer, effectuer l'opération. ok fait
```

[49]:
```python
#Visualisation des valeurs de la colonne sku
#Quelles sont les valeurs qui ne semblent pas respecter la régle de␣
↪codification?

## Voir les valeurs dont le type n'est pas integer
non_int_values = df_web_clean[df_web_clean['sku'].apply(lambda x: not␣
↪isinstance(x, int))]

## Afficher uniquement la colonne sku concernée
print(non_int_values[['sku']])
```

```
                      sku
8                     NaN
20                    NaN
30                    NaN
37                    NaN
41                    NaN
…                     …
1387  bon-cadeau-25-euros
1429                  NaN
1432                  NaN
1445                  NaN
1457                  NaN

[89 rows x 1 columns]
```

[50]:
```python
#Identifier les lignes sans code article
sku_nan = df_web_clean[df_web_clean['sku'].isna()]

# Afficher les lignes concernées
print(sku_nan[['sku']])
```

```
      sku
8     NaN
20    NaN
30    NaN
37    NaN
41    NaN
…     …
```

```
1384  NaN
1429  NaN
1432  NaN
1445  NaN
1457  NaN

[85 rows x 1 columns]
```

[51]:
```python
#Pour les codes articles identifiés, réaliser une analyse et définir l'action à␣
↪entreprendre
df_web_clean_nan = df_web_clean[df_web_clean['sku'].isna()]
df_web_clean_nan
```

[51]:
```
      sku  total_sales tax_status product_type post_name post_modified  \
8     NaN          NaN        NaN          NaN       NaN           NaT
20    NaN          NaN        NaN          NaN       NaN           NaT
30    NaN          NaN        NaN          NaN       NaN           NaT
37    NaN          NaN        NaN          NaN       NaN           NaT
41    NaN          NaN        NaN          NaN       NaN           NaT
…   …           …         …           …        …            …
1384  NaN          NaN        NaN          NaN       NaN           NaT
1429  NaN          NaN        NaN          NaN       NaN           NaT
1432  NaN          NaN        NaN          NaN       NaN           NaT
1445  NaN          NaN        NaN          NaN       NaN           NaT
1457  NaN          NaN        NaN          NaN       NaN           NaT

     post_type
8          NaN
20         NaN
30         NaN
37         NaN
41         NaN
…        …
1384       NaN
1429       NaN
1432       NaN
1445       NaN
1457       NaN

[85 rows x 7 columns]
```

[52]:
```python
from IPython.display import display, HTML

display(HTML("<h2><b>1) ANALYSE sku NaN : À première vue, les lignes sans code␣
↪SKU (NaN) sont également non renseignées dans les autres colonnes. Il faut␣
↪pousser l'analyse et voir si les autres colonnes contiennent des valeurs␣
↪autres que null.</b></h2>"))
```

```
<IPython.core.display.HTML object>
```

[53]:
```python
## afficher le nb de valeurs par colonne de df_web_clean_nan
df_web_clean_nan.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 85 entries, 8 to 1457
Data columns (total 7 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   sku            0 non-null      object
 1   total_sales    2 non-null      float64
 2   tax_status     2 non-null      object
 3   product_type   2 non-null      object
 4   post_name      2 non-null      object
 5   post_modified  2 non-null      datetime64[ns]
 6   post_type      2 non-null      object
dtypes: datetime64[ns](1), float64(1), object(5)
memory usage: 5.3+ KB
```

[54]:
```python
## Afficher les lignes qui ont AU MOINS une valeur différente de NaN
lignes_avec_data = df_web_clean_nan[df_web_clean_nan.notna().any(axis=1)]

# Afficher les lignes concernées
lignes_avec_data
```

[54]:
```
      sku  total_sales tax_status product_type  \
1084  NaN        -56.0    taxable          Vin
1087  NaN        -17.0    taxable          Vin

                                       post_name       post_modified post_type
1084    pierre-jean-villa-condrieu-suspendu-2018 2019-11-02 13:24:01   product
1087  pierre-jean-villa-cote-rotie-fongeant-2017 2019-11-02 13:24:15   product
```

[55]:
```python
### VERIF SUPPLEMENTAIRES des valeurs <> NaN dans chaque colonne
## TOTAL_SALES : afficher les valeurs de la colonne
unique_sales_sku_nan = df_web_clean_nan['total_sales'].unique()
print(unique_sales_sku_nan)
```

```
[ nan -56. -17.]
```

[56]:
```python
## TAX_STATUS : afficher les valeurs de la colonne
unique_tax_status_sku_nan = df_web_clean_nan['tax_status'].unique()
print(unique_tax_status_sku_nan)
```

```
[nan 'taxable']
```

[57]:
```python
## PRODUCT_TYPE : afficher les valeurs de la colonne
unique_product_type_sku_nan = df_web_clean_nan['product_type'].unique()
```

```
print(unique_product_type_sku_nan)
```

```
[nan 'Vin']
```

[58]:
```
## POST_NAME : afficher les valeurs de la colonne
unique_post_name_sku_nan = df_web_clean_nan['post_name'].unique()
print(unique_post_name_sku_nan)
```

```
[nan 'pierre-jean-villa-condrieu-suspendu-2018'
 'pierre-jean-villa-cote-rotie-fongeant-2017']
```

[59]:
```
## POST_TYPE : afficher les valeurs de la colonne
unique_post_type_sku_nan = df_web_clean_nan['post_type'].unique()
print(unique_post_type_sku_nan)
```

```
[nan 'product']
```

[60]:
```
## POST_MODIFIED : afficher les valeurs de la colonne
unique_post_modified_sku_nan = df_web_clean_nan['post_modified'].unique()
print(unique_post_modified_sku_nan)
```

```
<DatetimeArray>
['NaT', '2019-11-02 13:24:01', '2019-11-02 13:24:15']
Length: 3, dtype: datetime64[ns]
```

[61]:
```
display(HTML("<h2><b>2) ANALYSE sku non integer : isoler ces lignes et voir␣
↪s'il est possible de retrouver les id sku selon les valeurs des autres␣
↪colonnes.</b></h2>"))
```

```
<IPython.core.display.HTML object>
```

[62]:
```
## Extraire les valeurs uniques non entières
unique_non_int_sku = df_web_clean['sku'].dropna().unique()  # Enlève les NaN␣
↪d'abord

# Filtrer uniquement celles qui ne sont pas des int
unique_non_int_sku = [val for val in unique_non_int_sku if not isinstance(val,␣
↪int)]
unique_non_int_sku
```

[62]:
```
['13127-1', 'bon-cadeau-25-euros']
```

[63]:
```
## Afficher le df filtré sur ces 2 valeurs
sku_non_integer = df_web_clean[df_web_clean['sku'].isin(['13127-1',␣
↪'bon-cadeau-25-euros'])]
sku_non_integer
```

[63]:
```
                      sku  total_sales tax_status product_type  \
272                13127-1          4.0    taxable          Vin
842    bon-cadeau-25-euros          7.0        NaN        Autre
```

```
1117              13127-1          4.0        NaN          Vin
1387  bon-cadeau-25-euros          7.0    taxable          NaN

                                          post_name       post_modified  \
272   clos-du-mont-olivet-chateauneuf-du-pape-2007-2 2020-07-20 17:09:06
842                            bon-cadeau-de-25-euros 2018-06-01 14:13:57
1117  clos-du-mont-olivet-chateauneuf-du-pape-2007-2 2020-07-20 17:09:06
1387                           bon-cadeau-de-25-euros 2018-06-01 14:13:57

       post_type
272      product
842   attachment
1117  attachment
1387     product
```

[64]:
```
post_name_sku_non_integer = df_web_clean[df_web_clean['post_name'].
 ↪isin(['clos-du-mont-olivet-chateauneuf-du-pape-2007-2',␣
 ↪'bon-cadeau-de-25-euros'])]
post_name_sku_non_integer
```

[64]:
```
                     sku  total_sales tax_status product_type  \
272               13127-1          4.0    taxable          Vin
842   bon-cadeau-25-euros          7.0        NaN        Autre
1117              13127-1          4.0        NaN          Vin
1387  bon-cadeau-25-euros          7.0    taxable          NaN

                                          post_name       post_modified  \
272   clos-du-mont-olivet-chateauneuf-du-pape-2007-2 2020-07-20 17:09:06
842                            bon-cadeau-de-25-euros 2018-06-01 14:13:57
1117  clos-du-mont-olivet-chateauneuf-du-pape-2007-2 2020-07-20 17:09:06
1387                           bon-cadeau-de-25-euros 2018-06-01 14:13:57

       post_type
272      product
842   attachment
1117  attachment
1387     product
```

[65]:
```
display(HTML("<h2><b>3) Que faire avec ces id sku (NaN ayant des valeurs dans␣
 ↪les autres colonnes, et sku non integer)  -> prévenir le client de cette␣
 ↪anomalie</b></h2>"))
```

```
<IPython.core.display.HTML object>
```

[66]:
```
#La clé pour chaque ligne est-elle unique? autrement dit, y a-t-il des doublons?
sku_count = df_web_clean['sku'].value_counts()
sku_doublons = sku_count[sku_count > 1]
sku_doublons
```

[66]:
```
sku
11862    2
16324    2
3507     2
14773    2
15967    2
        ..
12639    2
15466    2
15184    2
13078    2
14569    2
Name: count, Length: 714, dtype: int64
```

[67]:
```python
## Afficher un df ne comprenant que les doublons

# 1: Exclure les lignes où SKU est NaN
df_valid_sku = df_web_clean[df_web_clean['sku'].notna()].copy()  #ajoute .
 ↪copy() pour plus de sécurité

# 2: Convertir tous les SKU en chaînes de caractères
df_valid_sku.loc[:, 'sku'] = df_valid_sku['sku'].astype(str)

# 3: Filtrer les lignes où le SKU est dupliqué
df_doublons = df_valid_sku[df_valid_sku.duplicated(subset='sku', keep=False)].
 ↪sort_values(by='sku', ascending=True)
df_doublons
```

[67]:
```
                      sku  total_sales tax_status product_type  \
668                 10014         10.0    taxable          Gin
1030                10014         10.0        NaN          Gin
887                 10459          4.0        NaN          Vin
748                 10459          4.0    taxable          Vin
802                 10775          6.0    taxable          Vin
…                   …           …         …           …
33                   9636          0.0    taxable       Whisky
167                  9937          2.0    taxable          Vin
728                  9937          2.0        NaN          Vin
842   bon-cadeau-25-euros          7.0        NaN        Autre
1387  bon-cadeau-25-euros          7.0    taxable          NaN

                                          post_name       post_modified  \
668                    darnleys-london-dry-gin-original 2020-08-26 15:55:02
1030                   darnleys-london-dry-gin-original 2020-08-26 15:55:02
887   alphonse-mellot-sancerre-rouge-generation-xix-… 2020-08-24 14:05:02
748   alphonse-mellot-sancerre-rouge-generation-xix-… 2020-08-24 14:05:02
802   albert-mann-pinot-gris-vendanges-tardives-alte… 2018-12-22 10:30:02
```

```
…                                                    …                   …
33                         whisky-wemyss-the-hive-12-ans 2020-08-19 17:45:03
167   marcel-richaud-cairanne-rouge-les-estrambords-… 2019-06-26 09:32:25
728   marcel-richaud-cairanne-rouge-les-estrambords-… 2019-06-26 09:32:25
842                                bon-cadeau-de-25-euros 2018-06-01 14:13:57
1387                               bon-cadeau-de-25-euros 2018-06-01 14:13:57

       post_type
668      product
1030  attachment
887   attachment
748      product
802      product
…          …
33       product
167      product
728   attachment
842   attachment
1387     product

[1428 rows x 7 columns]
```

[68]:
```python
## Afficher un df filtré sur post_type = attachment
df_doublons_attachment = df_doublons[df_doublons['post_type'] == 'attachment']
df_doublons_attachment
```

[68]:
```
                      sku  total_sales tax_status product_type  \
1030                10014         10.0        NaN          Gin
887                 10459          4.0        NaN          Vin
1317                10775          6.0        NaN          Vin
520                 10814          7.0        NaN          Vin
1322                11049          4.0        NaN          Vin
…                   …          …         …           …
463                  8463          2.0        NaN          Vin
494                  9562          5.0        NaN    Champagne
762                  9636          0.0        NaN       Whisky
728                  9937          2.0        NaN          Vin
842   bon-cadeau-25-euros          7.0        NaN        Autre

                                              post_name       post_modified  \
1030                   darnleys-london-dry-gin-original 2020-08-26 15:55:02
887   alphonse-mellot-sancerre-rouge-generation-xix-… 2020-08-24 14:05:02
1317  albert-mann-pinot-gris-vendanges-tardives-alte… 2018-12-22 10:30:02
520   thierry-germain-saumur-champigny-outre-terre-2013 2020-08-05 14:35:02
1322  alphonse-mellot-sancerre-rouge-en-grands-champ… 2020-01-31 09:30:02
…                                                    …                   …
463   domaine-huet-vouvray-le-mont-moelleux-premiere… 2020-05-09 16:29:10
```

```
494                 champagne-larmandier-bernier-latitude 2018-12-28 15:10:04
762                         whisky-wemyss-the-hive-12-ans 2020-08-19 17:45:03
728   marcel-richaud-cairanne-rouge-les-estrambords-… 2019-06-26 09:32:25
842                               bon-cadeau-de-25-euros 2018-06-01 14:13:57

       post_type
1030  attachment
887   attachment
1317  attachment
520   attachment
1322  attachment
…            …
463   attachment
494   attachment
762   attachment
728   attachment
842   attachment

[714 rows x 7 columns]
```

[69]:
```
## Vérification des valeurs uniques en tax_status
unique_tax_status_attachment_type = df_doublons_attachment['tax_status'].
 ↪unique()
print(unique_tax_status_attachment_type)
```

```
[nan]
```

[70]:
```
from IPython.display import display, HTML

html_text = """
<h2><b>4) ANALYSE DOUBLONS SKU : dans un df ne comprenant que des doublons sku,␣
 ↪chaque doublon a des valeurs identiques en product_type, post_name et␣
 ↪post_modified.<br>
Les lignes comprenant la valeur 'attachment' en post_type n'ont que des valeurs␣
 ↪nulles en tax_status.<br>
Nb de ligne df avec doublons = 1428. Nb de ligne df doublons filtré sur␣
 ↪post_type 'attachment' = 714.<br>
Demander validation du client et supprimer car non utile à l'analyse</b></h2>
"""

display(HTML(html_text))
```

```
<IPython.core.display.HTML object>
```

[71]:
```
## Supprimer les sku NaN, les sku String, les lignes en doublon␣
 ↪df_doublons_attachment
df_web_clean_vfinale1 = df_web_clean[df_web_clean['sku'].notnull()]
```

```
df_web_clean_vfinale2 = df_web_clean_vfinale1[df_web_clean_vfinale1['sku']!
 ↪='bon-cadeau-25-euros']
df_web_clean_vfinale3 =␣
 ↪df_web_clean_vfinale2[df_web_clean_vfinale2['post_type']=='product']
df_web_clean_vfinale3.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 713 entries, 2 to 1509
Data columns (total 7 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   sku            713 non-null    object
 1   total_sales    713 non-null    float64
 2   tax_status     713 non-null    object
 3   product_type   713 non-null    object
 4   post_name      713 non-null    object
 5   post_modified  713 non-null    datetime64[ns]
 6   post_type      713 non-null    object
dtypes: datetime64[ns](1), float64(1), object(5)
memory usage: 44.6+ KB
```

[72]:
```
non_int_values2 = df_web_clean_vfinale3[df_web_clean_vfinale3['sku'].
 ↪apply(lambda x: not isinstance(x, int))]
non_int_values2
```

[72]:
```
         sku  total_sales tax_status product_type  \
272  13127-1          4.0    taxable          Vin

                                          post_name       post_modified  \
272  clos-du-mont-olivet-chateauneuf-du-pape-2007-2 2020-07-20 17:09:06

    post_type
272   product
```

[73]:
```
#Les lignes sans code article semblent être toutes non renseignées
#Pour s'en assurer, réaliser les étapes suivantes:
#1 - Créer un dataframe avec uniquement les lignes sans code article

#2 - Utiliser la fonction df.info() sur ce nouveau dataframe pour observer le␣
 ↪nombre de valeurs renseignées dans chacune des colonnes

#3 - Que constatez-vous? -> Traité dans les cellules précédentes : lignes NaN␣
 ↪inutiles à l'analyse
```

2.3 - Analyse exploratoire du fichier liaison.xlsx

[75]:
```
#Dimension du dataset
#Nombre d'observations
```

```
print("Le tableau comporte {} observation(s) ou article(s)".format(df_liaison.
 ↪shape[0]))
#Nombre de caractéristiques
print("Le tableau comporte {} colonne(s)".format(df_liaison.shape[1]))
```

```
Le tableau comporte 825 observation(s) ou article(s)
Le tableau comporte 2 colonne(s)
```

[76]:
```
#Consulter le nombre de colonnes
#La nature des données dans chacune des colonnes
#Le nombre de valeurs présentes dans chacune des colonnes
df_liaison.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 825 entries, 0 to 824
Data columns (total 2 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   id_web      734 non-null    object
 1   product_id  825 non-null    int64
dtypes: int64(1), object(1)
memory usage: 13.0+ KB
```

[77]:
```
#Les valeurs de la colonne "product_id" sont-elles toutes uniques? OUI
product_id_count = df_liaison['product_id'].value_counts()
product_id_doublons = product_id_count[product_id_count > 1]
product_id_doublons
```

[77]:
```
Series([], Name: count, dtype: int64)
```

[78]:
```
#Les valeurs de la colonne "id_web" sont-elles toutes uniques? OUI
id_web_count = df_liaison['id_web'].value_counts()
id_web_doublons = id_web_count[id_web_count > 1]
id_web_doublons
```

[78]:
```
Series([], Name: count, dtype: int64)
```

[79]:
```
#Avons-nous des articles sans correspondance?
df_id_web_nan = df_liaison[df_liaison['id_web'].isna()]
df_id_web_nan
```

[79]:
```
     id_web  product_id
19      NaN        4055
49      NaN        4090
50      NaN        4092
119     NaN        4195
131     NaN        4209
..      …           …
```

```
817    NaN        7196
818    NaN        7200
819    NaN        7201
820    NaN        7203
821    NaN        7204

[91 rows x 2 columns]
```

[80]:
```python
df_liaison.nunique()
```

[80]:
```
id_web        734
product_id    825
dtype: int64
```

[81]:
```python
## Suppression des valeurs nulles NaN
df_liaison_clean = df_liaison[df_liaison['id_web'].notnull()]
df_liaison_clean.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 734 entries, 0 to 824
Data columns (total 2 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   id_web      734 non-null    object
 1   product_id  734 non-null    int64
dtypes: int64(1), object(1)
memory usage: 17.2+ KB
```

Etape 3 - Jonction des fichiers

Etape 3.1 - Jonction du fichier df_erp et df_liaison

[84]:
```python
#Fusion des fichiers df_erp et df_liaison
df_erp_liaison = df_liaison_clean.merge(df_erp, on='product_id', how='outer')
df_erp_liaison
```

[84]:
```
      id_web  product_id  onsale_web  price  stock_quantity stock_status  \
0      15298        3847           1   24.2              16      instock
1      15296        3849           1   34.3              10      instock
2      15300        3850           1   20.8               0   outofstock
3      19814        4032           1   14.1              26      instock
4      19815        4039           1   46.0               3      instock
..       …           …           …     …               …           …
820      NaN        7203           0   45.0              30      instock
821      NaN        7204           0   45.0               9      instock
822  13127-1        7247           1   54.8               6      instock
823  14680-1        7329           0   26.5              14      instock
824    16230        7338           1   16.3              40      instock
```

```
     purchase_price stock_status_2
0             12.88        instock
1             17.54        instock
2             10.64     outofstock
3              6.92        instock
4             23.77        instock
..              …            …
820           23.48        instock
821           24.18        instock
822           27.18        instock
823           13.42        instock
824            8.00        instock

[825 rows x 8 columns]
```

[85]:
```python
#Y a t-il des lignes ne "matchant" pas entre les 2 fichiers?
df_erp_liaison_check_corresp = df_liaison.merge(df_erp, on='product_id',␣
↪how='outer', indicator=True)
df_erp_liaison_check_corresp
```

[85]:
```
      id_web  product_id  onsale_web  price  stock_quantity stock_status  \
0      15298        3847           1   24.2              16      instock
1      15296        3849           1   34.3              10      instock
2      15300        3850           1   20.8               0   outofstock
3      19814        4032           1   14.1              26      instock
4      19815        4039           1   46.0               3      instock
..       …           …          …     …                 …           …
820      NaN        7203           0   45.0              30      instock
821      NaN        7204           0   45.0               9      instock
822  13127-1        7247           1   54.8               6      instock
823  14680-1        7329           0   26.5              14      instock
824    16230        7338           1   16.3              40      instock

     purchase_price stock_status_2 _merge
0             12.88        instock   both
1             17.54        instock   both
2             10.64     outofstock   both
3              6.92        instock   both
4             23.77        instock   both
..              …            …     …
820           23.48        instock   both
821           24.18        instock   both
822           27.18        instock   both
823           13.42        instock   both
824            8.00        instock   both

[825 rows x 9 columns]
```

[86]:
```
## Vérification du nb de lignes ayant été appariées
df_erp_liaison_check_corresp['_merge'].value_counts()
```

[86]:
```
_merge
both          825
left_only       0
right_only      0
Name: count, dtype: int64
```

[87]:
```
## Vérification du nb de product_id distincts df_erp
nb_product_id_uniques = df_erp['product_id'].nunique()
print(f"Nombre de product_id distincts dans df_erp[product_id] :␣
↪{nb_product_id_uniques}")
```

```
Nombre de product_id distincts dans df_erp[product_id] : 825
```

Etape 3.2 - Jonction du fichier df_erp_liaison et df_web

[89]:
```
#Fusionner les datasets df_erp_liaison et df_web
# 1 : renommer df_web[sku] en [id_web]
df_web_clean_vfinale3_new = df_web_clean_vfinale3.rename(columns={'sku':
↪'id_web'})

# 2 : fusion
df_merge = df_erp_liaison.merge(df_web_clean_vfinale3_new, on='id_web',␣
↪how='outer')
```

[90]:
```
#Avons-nous des lignes sans correspondance?
df_merge_check_corresp = df_erp_liaison.merge(df_web_clean_vfinale3_new,␣
↪on='id_web', how='outer', indicator=True)
df_merge_check_corresp['_merge'].value_counts()
```

[90]:
```
_merge
both          713
left_only     112
right_only      0
Name: count, dtype: int64
```

[91]:
```
# Filtrer df sur valeur 'both' de la colonne _merge
df_merge_vf = df_merge_check_corresp[df_merge_check_corresp['_merge']=='both']
df_merge_vf.info()
```

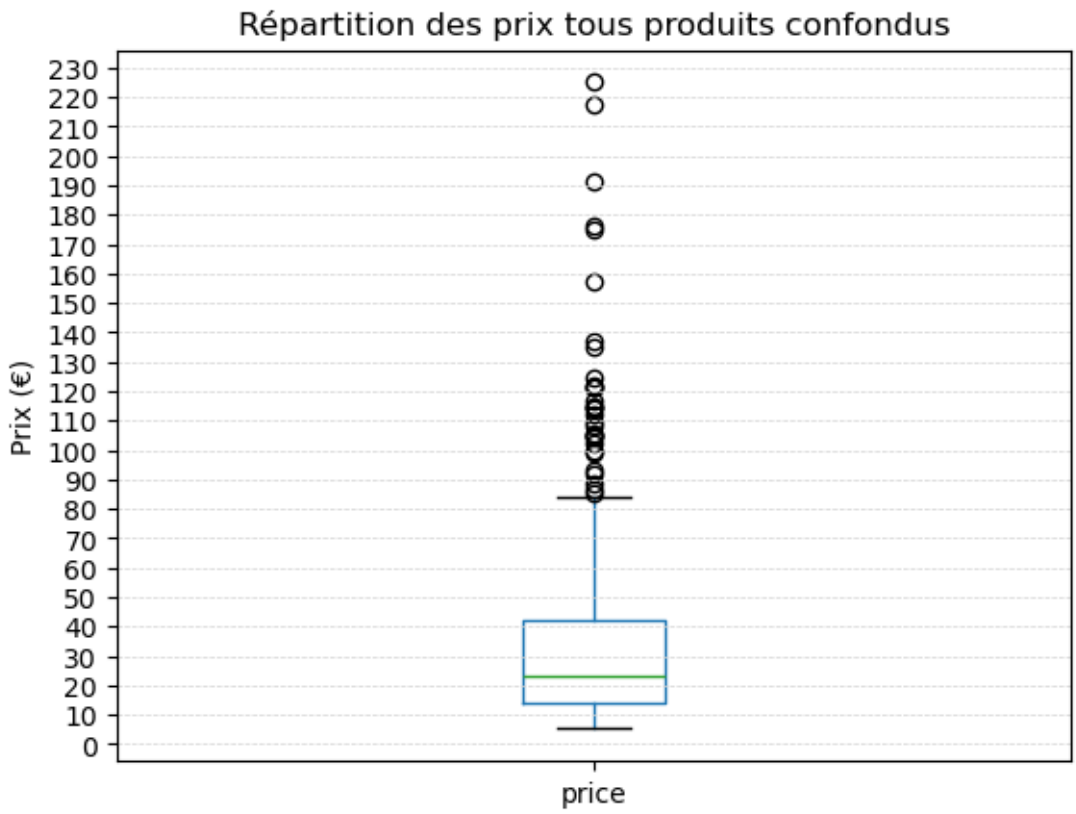
```
<class 'pandas.core.frame.DataFrame'>
Index: 713 entries, 0 to 731
Data columns (total 15 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   id_web          713 non-null    object
 1   product_id      713 non-null    int64
```

```
 2   onsale_web      713 non-null    int64
 3   price           713 non-null    float64
 4   stock_quantity  713 non-null    int64
 5   stock_status    713 non-null    object
 6   purchase_price  713 non-null    float64
 7   stock_status_2  713 non-null    object
 8   total_sales     713 non-null    float64
 9   tax_status      713 non-null    object
 10  product_type    713 non-null    object
 11  post_name       713 non-null    object
 12  post_modified   713 non-null    datetime64[ns]
 13  post_type       713 non-null    object
 14  _merge          713 non-null    category
dtypes: category(1), datetime64[ns](1), float64(3), int64(3), object(7)
memory usage: 84.4+ KB
```

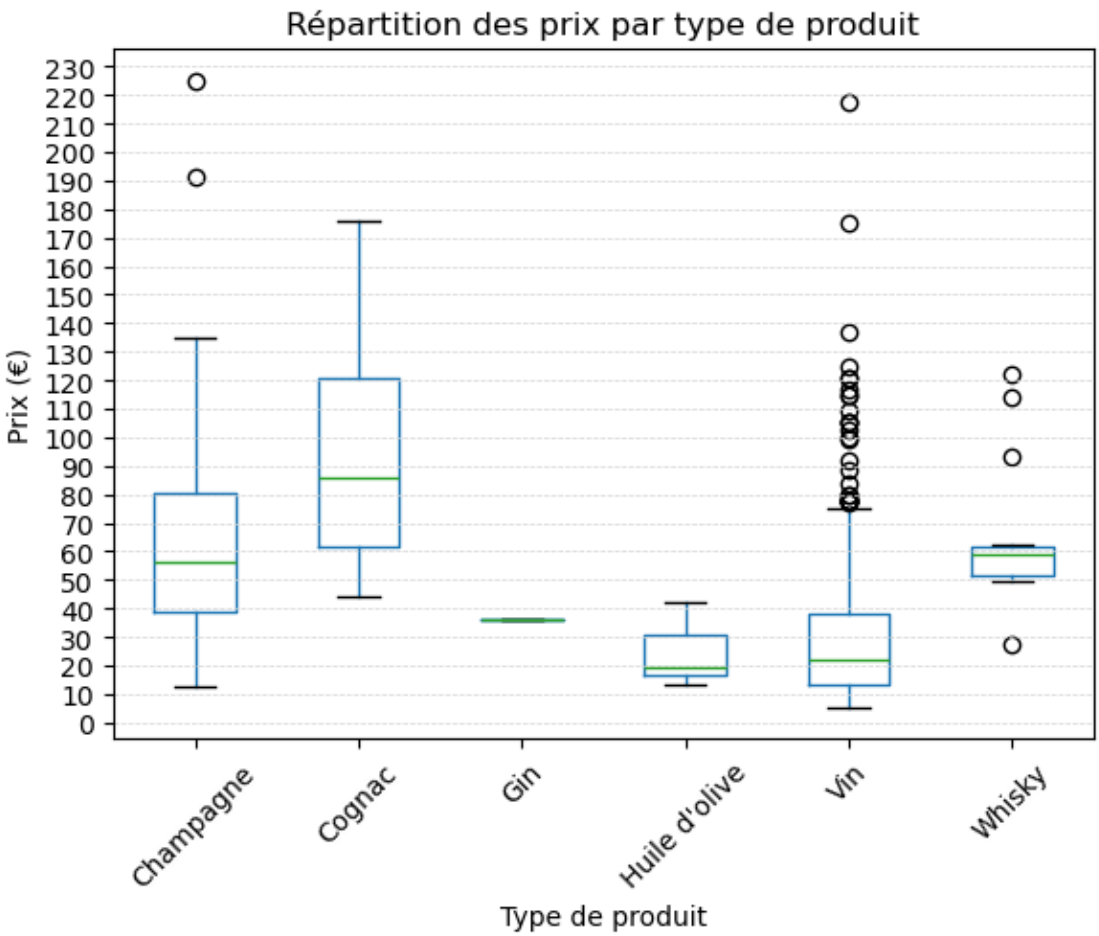
Etape 4 - Analyse univariée des prix

Etape 4.1 - Exploration par la visualisation de données

[94]:
```
#Création d'une boîte à moustache de la répartition des prix grâce à Pandas

# 1. Création du boxplot
df_merge_vf.boxplot(column='price', grid=False)

# 2. Personnalisation
plt.title("Répartition des prix tous produits confondus")
plt.ylabel("Prix (€)")

# 3. Ajout des lignes horizontales tous les 10 €
max_price = df_merge_vf['price'].max()
plt.yticks(range(0, int(max_price) + 10, 10))  # Marqueurs tous les 10 €
for y in range(0, int(max_price) + 10, 10):
    plt.axhline(y, color='lightgray', linestyle='--', linewidth=0.5)

plt.show()
```



[95]:
```
## Exploration supplémentaire : boite à moustache par type de produit
# 1. Création du boxplot par type de produit
df_merge_vf.boxplot(column='price', by='product_type', grid=False)

# 2. Personnalisation
plt.title("Répartition des prix par type de produit")
plt.suptitle("")  # Supprime le titre automatique
plt.xlabel("Type de produit")
plt.ylabel("Prix (€)")
plt.xticks(rotation=45)

# 3. Ajout des lignes horizontales tous les 10 €
max_price = df_merge_vf['price'].max()
plt.yticks(range(0, int(max_price) + 10, 10))  # Marqueurs tous les 10 €
for y in range(0, int(max_price) + 10, 10):
    plt.axhline(y, color='lightgray', linestyle='--', linewidth=0.5)

plt.show()
```



[96]:
```
#Autre méthode avec plotly express
import plotly.express as px

fig = px.box(
    df_merge_vf,
    y='price',
    points='outliers',
    hover_data=['product_type'],  # Ajoute le type de produit dans l'infobulle
    title='Répartition des prix tous produits confondus'
)

fig.update_layout(
    yaxis_title='Prix (€)',
    boxmode='group',
    template='plotly_white'
)
```

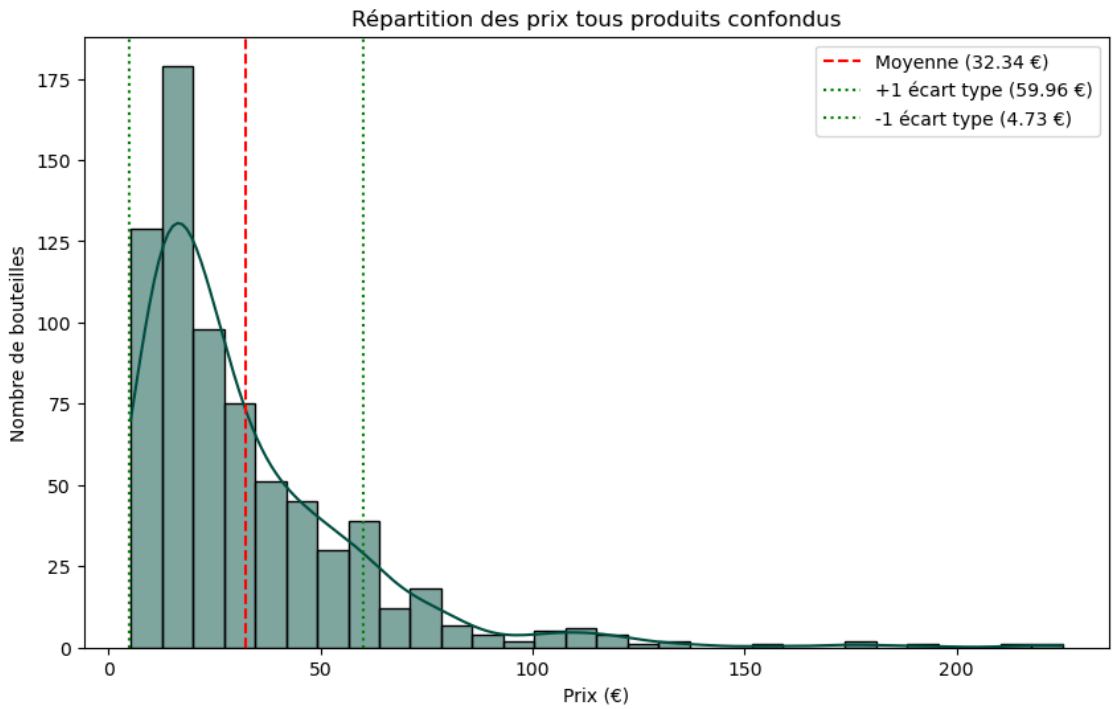
```
fig.show()
```

Etape 4.2 - Exploration par l'utilisation de méthodes statistiques

Etape 4.2.1 - Identification par le Z-index

[99]:
```
#Calculer la moyenne du prix
prix_moyen = df_merge_vf['price'].mean()
print(f"Le prix moyen est de {prix_moyen:.2f} euros.")

#Calculer l'écart-type du prix
ecart_type = df_merge_vf['price'].std()
print(f"L'écart type du prix est de {ecart_type:.2f} euros.")

# Dataviz écart type
import seaborn as sns

# Calculs
prix_moyen = df_merge_vf['price'].mean()
ecart_type = df_merge_vf['price'].std()

# Histogramme
plt.figure(figsize=(10, 6))
sns.histplot(df_merge_vf['price'], bins=30, kde=True, color='#004D40')  #␣
↪couleur modifiée ici

# Lignes de moyenne et écart type
plt.axvline(prix_moyen, color='red', linestyle='--', label=f'Moyenne␣
↪({prix_moyen:.2f} €)')
plt.axvline(prix_moyen + ecart_type, color='green', linestyle=':', label=f'+1␣
↪écart type ({(prix_moyen + ecart_type):.2f} €)')
plt.axvline(prix_moyen - ecart_type, color='green', linestyle=':', label=f'-1␣
↪écart type ({(prix_moyen - ecart_type):.2f} €)')

# Mise en forme
plt.title("Répartition des prix tous produits confondus")
plt.xlabel("Prix (€)")
plt.ylabel("Nombre de bouteilles")
plt.legend()
plt.show()
```

```
Le prix moyen est de 32.34 euros.
L'écart type du prix est de 27.61 euros.
```



[100]:
```
#Calculer le Z-score
# Calcul du Z-score pour chaque prix
df_merge_vf['z_score_price'] = (df_merge_vf['price'] - df_merge_vf['price'].
 ↪mean()) / df_merge_vf['price'].std()

# Aperçu des résultats
print(df_merge_vf[['post_name', 'price', 'z_score_price']].head())
```

```
                                          post_name  price  z_score_price
0         emile-boeckel-cremant-brut-blanc-de-blancs    8.6      -0.859842
1     marcel-windholtz-eau-de-marc-de-gewurztraminer   41.0       0.313461
2                    gosset-champagne-grande-reserve   39.0       0.241035
3                       cognac-normandin-mercier-vfc   59.9       0.997889
4  champagne-petit-lebrun-fils-blanc-de-blancs-gr…   22.5      -0.356480

C:\Users\t_rak\AppData\Local\Temp\ipykernel_5112\1348604367.py:3:
SettingWithCopyWarning:


A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-
docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
```

[101]:
```
z_score_3 = df_merge_vf[df_merge_vf['z_score_price']>3]
z_score_3.head()
```

[101]:
```
     id_web  product_id  onsale_web  price  stock_quantity stock_status  \
24     3510        4402           1  176.0              11      instock
33     7819        4406           1  157.0              12      instock
171   14220        4904           1  137.0               9      instock
204   14581        5001           1  217.5              18      instock
235   14775        5917           1  122.0              12      instock

      purchase_price stock_status_2  total_sales tax_status product_type  \
24             78.25        instock          3.0    taxable       Cognac
33             69.08        instock          4.0    taxable       Cognac
171            67.95        instock          3.0    taxable          Vin
204           116.87        instock          2.0    taxable          Vin
235            54.24        instock          3.0    taxable       Whisky

                                                post_name       post_modified  \
24                                   cognac-frapin-vip-xo 2020-08-22 11:35:03
33        cognac-frapin-chateau-de-fontpinot-1989-20-ans 2020-03-14 16:05:04
171  domaine-des-croix-corton-charlemagne-grand-cru… 2020-05-19 17:15:02
204       david-duband-charmes-chambertin-grand-cru-2014 2020-05-16 09:00:05
235  wemyss-malts-single-cask-scotch-whisky-choc-n-… 2020-03-11 09:30:09

     post_type _merge  z_score_price
24    product   both       5.202227
33    product   both       4.514178
171   product   both       3.789917
204   product   both       6.705069
235   product   both       3.246721
```

[102]:
```
z_score_3.shape
```

[102]: (13, 16)

[103]:
```
#Quel est le seuil prix dont le z-score est supérieur à 3?
z_score_3.sort_values(by='z_score_price', ascending=True, inplace=True)
z_score_3[['post_name', 'price', 'z_score_price']].head(1)
print(f"Le seuil prix dont le z score est >3 est {z_score_3['price'].head(1).
 ↪values} euros.")
```

```
Le seuil prix dont le z score est >3 est [116.4] euros.

C:\Users\t_rak\AppData\Local\Temp\ipykernel_5112\3312476417.py:2:
SettingWithCopyWarning:


A value is trying to be set on a copy of a slice from a DataFrame
```

```
See the caveats in the documentation: https://pandas.pydata.org/pandas-
docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
```

Etape 4.2.2 - Identification par l'intervalle interquartile

[105]:
```
#Utilisation de la fonction "describe" de Pandas pour l'étude des mesures de␣
↪dispersion
df_merge_vf['price'].describe()
```

[105]:
```
count    713.000000
mean      32.343969
std       27.614335
min        5.200000
25%       14.050000
50%       23.400000
75%       42.100000
max      225.000000
Name: price, dtype: float64
```

[106]:
```
#Définir un seuil pour les articles "outliers" en prix
Q1 = df_merge_vf['price'].quantile(0.25)
Q3 = df_merge_vf['price'].quantile(0.75)
IQR = Q3 - Q1

seuil_iqr = Q3 + 1.5 * IQR
print(f"Seuil IQR pour les outliers : {seuil_iqr:.2f} €")
print(f"Interprétation : Tout produit dont le prix est supérieur à {seuil_iqr:.
↪2f} €, est considéré comme extêmement cher par rapport au reste du␣
↪catalogue")
```

```
Seuil IQR pour les outliers : 84.18 €
Interprétation : Tout produit dont le prix est supérieur à 84.18 €, est
considéré comme extêmement cher par rapport au reste du catalogue
```

[107]:
```
#Définir le nombre d'articles et la proportion de l'ensemble du catalogue␣
↪"outliers"
outliers_iqr = df_merge_vf[df_merge_vf['price'] > seuil_iqr]
nb_outliers_iqr = outliers_iqr.shape[0]
proportion_iqr = nb_outliers_iqr / df_merge_vf.shape[0] * 100

print(f"Nombre d'articles outliers (IQR) : {nb_outliers_iqr}")
print(f"Proportion du catalogue : {proportion_iqr:.2f} %")
```

```
Nombre d'articles outliers (IQR) : 31
Proportion du catalogue : 4.35 %
```

[108]:
```
#Selon vous, ces outliers sont-ils justifiés ? Comment le démontrer si cela est␣
↪possible ?
```

```
outliers_iqr[['post_name', 'price', 'product_type', 'stock_quantity',␣
↪'total_sales']].sort_values(by='product_type')
```

[108]:

```
                                                post_name  price product_type  \
257                 champagne-gosset-celebris-vintage-2007  135.0    Champagne
150  champagne-agrapart-fils-lavizoise-grand-cru-20…  112.0    Champagne
149  champagne-agrapart-fils-mineral-extra-brut-bla…   86.8    Champagne
567        champagne-egly-ouriet-grand-cru-millesime-2008  225.0    Champagne
269  coteaux-champenois-egly-ouriet-ambonnay-rouge-…  191.3    Champagne
143  champagne-larmandier-bernier-grand-cru-vieille…   85.6    Champagne
22                            cognac-frapin-fontpinot-xo  108.5       Cognac
23                             cognac-frapin-cigar-blend  104.0       Cognac
24                                  cognac-frapin-vip-xo  176.0       Cognac
33        cognac-frapin-chateau-de-fontpinot-1989-20-ans  157.0       Cognac
256  domaine-weinbach-gewurztraminer-gc-furstentum-…  124.8          Vin
281  domaine-des-comtes-lafon-volnay-1er-cru-champa…  121.0          Vin
291           domaine-clerget-echezeaux-en-orveaux-2015  116.4          Vin
313                 camille-giroud-clos-de-vougeot-2016  175.0          Vin
370  zind-humbrecht-riesling-gc-rangen-thann-clos-s…  100.0          Vin
282  domaine-des-comtes-lafon-volnay-1er-cru-santen…  121.0          Vin
240  domaine-des-croix-corton-grand-cru-les-greves-…  102.3          Vin
204        david-duband-charmes-chambertin-grand-cru-2014  217.5          Vin
205  david-duband-chambolle-musigny-1er-cru-les-sen…  105.6          Vin
171  domaine-des-croix-corton-charlemagne-grand-cru…  137.0          Vin
157  domaine-des-comtes-lafon-volnay-1er-cru-santen…  115.0          Vin
89   chateau-de-puligny-montrachet-1cru-champ-canet…  109.6          Vin
88   domaine-des-comtes-lafon-volnay-1er-cru-santen…  105.0          Vin
87   domaine-des-comtes-lafon-volnay-1er-cru-champa…  115.0          Vin
56   zind-humbrecht-pinot-gris-grand-cru-rangen-de-…   88.4          Vin
54   domaine-des-comtes-lafon-volnay-1er-cru-santen…  105.0          Vin
53   domaine-des-comtes-lafon-volnay-1er-cru-champa…   99.0          Vin
730                        tempier-bandol-cabassaou-2017   92.0          Vin
234         wemyss-malts-single-cask-chocolate-moka-cake   93.0       Whisky
235  wemyss-malts-single-cask-scotch-whisky-choc-n-…  122.0       Whisky
233  wemyss-malts-single-cask-scotch-whisky-chai-ca…  114.0       Whisky

     stock_quantity  total_sales
257             138          5.0
150             136          6.0
149             101          9.0
567               0         11.0
269              98          6.0
143             112          7.0
22               17          4.0
23               14          5.0
24               11          3.0
33               12          4.0
```

```
256              19          1.0
281              14          2.0
291              12          5.0
313              12          4.0
370              12          1.0
282               9          3.0
240              12          2.0
204              18          2.0
205              16          6.0
171               9          3.0
157              16          7.0
89               18          1.0
88               15          3.0
87               14          4.0
56                7          5.0
54               12          7.0
53                9          6.0
730               0          1.0
234               1          1.0
235              12          3.0
233              12          3.0
```

[109]:
```
print("La majorité des outliers sont justifiés par leur positionnement produit :
 ↪ grands crus, spiritueux rares, champagnes millésimés.")
print("Quelques vins à prix élevé avec peu de ventes ou stock nul méritent une␣
 ↪vérification commerciale : erreur de saisie ? produit obsolète ? mauvaise␣
 ↪catégorisation ?")
```

```
La majorité des outliers sont justifiés par leur positionnement produit : grands
crus, spiritueux rares, champagnes millésimés.
Quelques vins à prix élevé avec peu de ventes ou stock nul méritent une
vérification commerciale : erreur de saisie ? produit obsolète ? mauvaise
catégorisation ?
```

[110]:
```
# 1. Filtre vins
vins = df_merge_vf[df_merge_vf['product_type'] == 'Vin'].copy()

# 2. Calculer le ratio stock / ventes
vins['stock_vs_sales_ratio'] = vins['stock_quantity'] / (vins['total_sales'] +␣
 ↪1e-6)  # évite division par zéro

# 3. Filtrer les cas où le stock est beaucoup plus élevé que les ventes (ratio␣
 ↪> 5 ?)
vins_surstock = vins[vins['stock_vs_sales_ratio'] > 5]

# 4. Afficher les résultats
print("Vins en surstock (stock >> ventes) :")
```

```
vins_surstock[['post_name', 'price', 'stock_quantity', 'total_sales',␣
↪'stock_vs_sales_ratio']]
```

```
Vins en surstock (stock >> ventes) :
```

[110]:

```
                                                post_name  price  stock_quantity  \
89   chateau-de-puligny-montrachet-1cru-champ-canet…  109.6              18
204     david-duband-charmes-chambertin-grand-cru-2014  217.5              18
240  domaine-des-croix-corton-grand-cru-les-greves-…  102.3              12
256  domaine-weinbach-gewurztraminer-gc-furstentum-…  124.8              19
281  domaine-des-comtes-lafon-volnay-1er-cru-champa…  121.0              14
370  zind-humbrecht-riesling-gc-rangen-thann-clos-s…  100.0              12

     total_sales  stock_vs_sales_ratio
89           1.0             17.999982
204          2.0              8.999996
240          2.0              5.999997
256          1.0             18.999981
281          2.0              6.999997
370          1.0             11.999988
```

Etape 5 - Analyse univariée du CA, des quantités vendues, des stocks et de la marge ainsi qu'une analyse multivariée

Etape 5.1 - Analyse des ventes en CA

[113]:

```
##############################
# Calculer le CA du site web #
##############################

#Créer une colonne calculant le CA par article
##Création de la colonne 'ca_par_article' = prix unitaire × nombre d'unités␣
↪vendues
df_merge_vf['ca_article'] = df_merge_vf['price'] * df_merge_vf['total_sales']

#Calculer la somme de la colonne "ca_par_article"
ca_total = df_merge_vf['ca_article'].sum()
print(f"Chiffre d'affaires total du catalogue : {ca_total:.2f} €")

#Ce résultat correspond au chiffre d'affaire du site web
```

```
Chiffre d'affaires total du catalogue : 143505.10 €
```

```
C:\Users\t_rak\AppData\Local\Temp\ipykernel_5112\2773008908.py:7:
SettingWithCopyWarning:


A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

[114]:
```
##############################
# Palmarès des articles en CA #
##############################

#Effectuer le tri dans l'ordre décroissant du CA du dataset df_merge
df_merge_vf.sort_values(by='ca_article', ascending=False, inplace=True)

#Réinitialiser l'index du dataset par un reset_index
df_merge_vf.reset_index(drop=True, inplace=True)

#Afficher les 20 premiers articles en CA
df_merge_vf[['post_name', 'price', 'total_sales', 'ca_article']].head(20)
```

```
C:\Users\t_rak\AppData\Local\Temp\ipykernel_5112\3295267628.py:6:
SettingWithCopyWarning:


A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-
docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
```

[114]:

	post_name	price	total_sales	\
0	champagne-egly-ouriet-grand-cru-millesime-2008	225.0	11.0	
1	coteaux-champenois-egly-ouriet-ambonnay-rouge-…	191.3	6.0	
2	champagne-egly-ouriet-grand-cru-brut-rose	79.5	14.0	
3	agnes-levet-amethyste-2017	41.2	20.0	
4	domaine-des-comtes-lafon-volnay-1er-cru-santen…	115.0	7.0	
5	champagne-agrapart-fils-mineral-extra-brut-bla…	86.8	9.0	
6	domaine-des-comtes-lafon-volnay-1er-cru-santen…	105.0	7.0	
7	camille-giroud-clos-de-vougeot-2016	175.0	4.0	
8	champagne-gosset-celebris-vintage-2007	135.0	5.0	
9	champagne-agrapart-fils-lavizoise-grand-cru-20…	112.0	6.0	
10	david-duband-chambolle-musigny-1er-cru-les-sen…	105.6	6.0	
11	cognac-frapin-chateau-de-fontpinot-1989-20-ans	157.0	4.0	
12	bernard-baudry-chinon-rouge-croix-boissee-2017	28.5	22.0	
13	champagne-larmandier-bernier-vieilles-vignes-l…	77.0	8.0	
14	champagne-larmandier-bernier-grand-cru-vieille…	85.6	7.0	
15	domaine-des-comtes-lafon-volnay-1er-cru-champa…	99.0	6.0	
16	domaine-clerget-echezeaux-en-orveaux-2015	116.4	5.0	
17	champagne-egly-ouriet-extra-brut-vp	79.5	7.0	
18	cdme-beaune-1ercru-greves-3-journaux-2015	67.2	8.0	
19	cognac-frapin-vip-xo	176.0	3.0	

```
    ca_article
0       2475.0
1       1147.8
2       1113.0
3        824.0
4        805.0
5        781.2
6        735.0
7        700.0
8        675.0
9        672.0
10       633.6
11       628.0
12       627.0
13       616.0
14       599.2
15       594.0
16       582.0
17       556.5
18       537.6
19       528.0
```

[115]:

```python
#Graphique en barre des 20 premiers articles avec plotly express
# 1. Sélection et tri des 20 premiers articles par CA croissant
top_20 = df_merge_vf.head(20).sort_values(by='ca_article', ascending=True)

# 2. Création du graphique avec axes inversés
fig = px.bar(
    top_20,
    x='ca_article',
    y='post_name',
    title='Top 20 des articles par chiffre d'affaires',
    labels={'post_name': 'Article', 'ca_article': 'Chiffre d'affaires (€)'},
    hover_data=['price', 'total_sales'],
    text='ca_article',
    orientation='h',
    color_discrete_sequence=['#004D40']
)

# 3. Personnalisation
fig.update_traces(
    texttemplate='%{text:.2f}',
    textposition='inside',
    marker_color='#004D40',
    textfont=dict(color='white')
)
```

```
fig.update_layout(
    template='plotly_white',
    height=900,
    width=1000,
    title_font=dict(color='#004D40', size=22),
    title_x=0.5,
    xaxis_title='Chiffre d'affaires (€)',
    yaxis_title=None,  # Masque le titre de l'axe Y
    xaxis=dict(title_font=dict(color='#004D40'),␣
 ↪tickfont=dict(color='#004D40')),
    yaxis=dict(tickfont=dict(color='#004D40')),
)

fig.show()
```

[116]:
```
#############################
# Calculer le 20 / 80 en CA #
#############################

#Créer une colonne calculant la part du CA de la ligne dans le dataset
df_merge_vf['part_ca'] = (df_merge_vf['ca_article'] / ca_total) * 100

#Créer une colonne réalisant la somme cumulative de la colonne précedemment␣
 ↪créée
# Création de la colonne 'part_ca_cumulee'
df_merge_vf['part_ca_cumulee'] = df_merge_vf['part_ca'].cumsum()
df_merge_vf[['post_name', 'part_ca', 'part_ca_cumulee']].head(10)

#Grâce aux deux colonnes créées précedemment, calculer le nombre d'articles␣
 ↪représentant 80% du CA
#1. Filtrer les articles dont la part cumulée est <= 80 %
articles_80_ca = df_merge_vf[df_merge_vf['part_ca_cumulee'] <= 80]

#2. Nombre d'articles concernés
nb_articles_80_ca = articles_80_ca.shape[0]
print(f"Nombre d'articles représentant 80 % du chiffre d'affaires :␣
 ↪{nb_articles_80_ca}")

#Afficher la proportion que représente ce groupe d'articles dans le catalogue␣
 ↪entier du site web
#1. Nombre total d'articles dans le catalogue
total_articles = df_merge_vf.shape[0]

#2. Proportion en pourcentage
proportion_80_ca = (nb_articles_80_ca / total_articles) * 100
```

```
print(f"Ces {nb_articles_80_ca} articles représentent {proportion_80_ca:.2f} %␣
↪du catalogue total.")
```

```
C:\Users\t_rak\AppData\Local\Temp\ipykernel_5112\18200316.py:6:
SettingWithCopyWarning:


A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-
docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

C:\Users\t_rak\AppData\Local\Temp\ipykernel_5112\18200316.py:10:
SettingWithCopyWarning:


A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-
docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy


Nombre d'articles représentant 80 % du chiffre d'affaires : 433
Ces 433 articles représentent 60.73 % du catalogue total.
```

Etape 5.2 - Analyse des ventes en quantité

[118]:
```
#####################################
# Palmarès des articles en quantité #
#####################################

#Effectuer le tri dans l'ordre décroissant de quantités vendues du dataset␣
↪df_merge
df_merge_vf.sort_values(by='total_sales', ascending=False, inplace=True)

#Réinitialiser l'index du dataset par un reset_index
df_merge_vf.reset_index(drop=True, inplace=True)

#Afficher les 20 premiers articles en quantité
df_merge_vf[['post_name', 'price', 'total_sales', 'ca_article']].head(20)
```

```
C:\Users\t_rak\AppData\Local\Temp\ipykernel_5112\3912337390.py:6:
SettingWithCopyWarning:


A value is trying to be set on a copy of a slice from a DataFrame
```

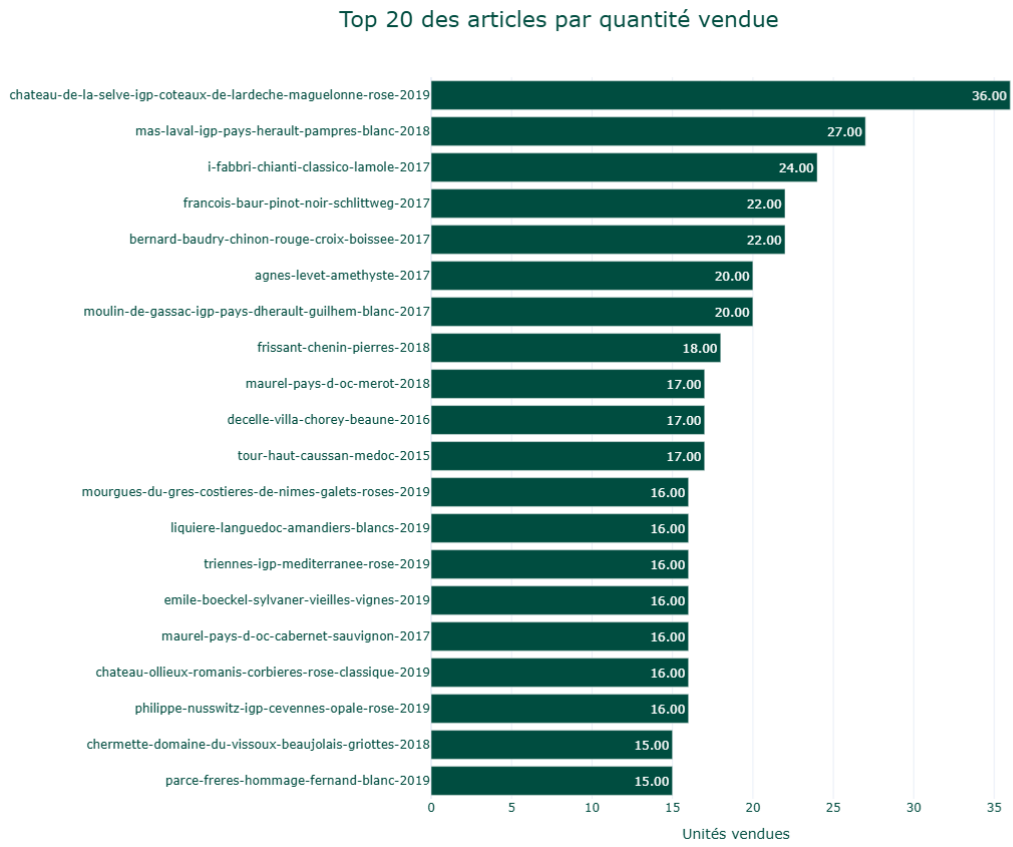
```
See the caveats in the documentation: https://pandas.pydata.org/pandas-
docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
```

[118]:

	post_name	price	total_sales	\
0	chateau-de-la-selve-igp-coteaux-de-lardeche-ma…	9.9	36.0	
1	mas-laval-igp-pays-herault-pampres-blanc-2018	9.9	27.0	
2	i-fabbri-chianti-classico-lamole-2017	14.9	24.0	
3	francois-baur-pinot-noir-schlittweg-2017	12.7	22.0	
4	bernard-baudry-chinon-rouge-croix-boissee-2017	28.5	22.0	
5	agnes-levet-amethyste-2017	41.2	20.0	
6	moulin-de-gassac-igp-pays-dherault-guilhem-bla…	5.2	20.0	
7	frissant-chenin-pierres-2018	11.6	18.0	
8	maurel-pays-d-oc-merot-2018	5.8	17.0	
9	decelle-villa-chorey-beaune-2016	29.0	17.0	
10	tour-haut-caussan-medoc-2015	17.1	17.0	
11	mourgues-du-gres-costieres-de-nimes-galets-ros…	8.7	16.0	
12	liquiere-languedoc-amandiers-blancs-2019	9.5	16.0	
13	triennes-igp-mediterranee-rose-2019	9.3	16.0	
14	emile-boeckel-sylvaner-vieilles-vignes-2019	6.8	16.0	
15	maurel-pays-d-oc-cabernet-sauvignon-2017	5.7	16.0	
16	chateau-ollieux-romanis-corbieres-rose-classiq…	8.2	16.0	
17	philippe-nusswitz-igp-cevennes-opale-rose-2019	6.5	16.0	
18	chermette-domaine-du-vissoux-beaujolais-griott…	9.9	15.0	
19	parce-freres-hommage-fernand-blanc-2019	8.9	15.0	

	ca_article
0	356.4
1	267.3
2	357.6
3	279.4
4	627.0
5	824.0
6	104.0
7	208.8
8	98.6
9	493.0
10	290.7
11	139.2
12	152.0
13	148.8
14	108.8
15	91.2
16	131.2
17	104.0
18	148.5
19	133.5

[119]:
```
#Graphique en barre des 20 premiers articles avec plotly express
# 1. Sélection et tri des 20 premiers articles
top_20 = df_merge_vf.head(20).sort_values(by='total_sales', ascending=True)

# 2. Création du graphique avec axes inversés
fig = px.bar(
    top_20,
    x='total_sales',
    y='post_name',
    title='Top 20 des articles par quantité vendue',
    labels={'post_name': 'Article', 'total_sales': 'Unités vendues'},
    hover_data=['price', 'ca_article', 'part_ca'],
    text='total_sales',
    orientation='h',
    color_discrete_sequence=['#004D40']
)

# 3. Personnalisation
fig.update_traces(
    texttemplate='%{text:.2f}',
    textposition='inside',
    marker_color='#004D40',
    textfont=dict(color='white')
)

fig.update_layout(
    template='plotly_white',
    height=900,
    title_font=dict(color='#004D40', size=22),
    title_x=0.5,  # Centre le titre
    xaxis_title='Unités vendues',
    yaxis_title=None,
    xaxis=dict(title_font=dict(color='#004D40'),␣
 ↪tickfont=dict(color='#004D40')),
    yaxis=dict(title_font=dict(color='#004D40'),␣
 ↪tickfont=dict(color='#004D40')),
)

fig.show()
```



[120]:
```
##############################
# Calculer le 20 / 80 en CA #
##############################

#Créer une colonne calculant la part en quantité de la ligne dans le dataset
#1. Calcul du total des unités vendues
total_quantite = df_merge_vf['total_sales'].sum()

#2. Création de la colonne 'part_quantite' en pourcentage
df_merge_vf['part_quantite'] = (df_merge_vf['total_sales'] / total_quantite) *␣
↪100

#Créer une colonne réalisant la somme cumulative de la colonne précedemment␣
↪créée
df_merge_vf['part_quantite_cumulee'] = df_merge_vf['part_quantite'].cumsum()

#Grâce aux deux colonnes créées précedemment, calculer le nombre d'articles␣
↪représentant 80% des ventes en quantité
```

```python
#1. Filtrer les articles dont la part cumulée est <= 80 %
articles_80_quantite = df_merge_vf[df_merge_vf['part_quantite_cumulee'] <= 80]

#2. Nombre d'articles concernés
nb_articles_80_quantite = articles_80_quantite.shape[0]
print(f"Nombre d'articles représentant 80 % des ventes en quantité :␣
↪{nb_articles_80_quantite}")

#Afficher la proportion que représente ce groupe d'articles dans le catalogue␣
↪entier du site web
#1. Nombre total d'articles dans le catalogue
total_articles = df_merge_vf.shape[0]

#2. Proportion en pourcentage
proportion_80_quantite = (nb_articles_80_quantite / total_articles) * 100

print(f"Ces {nb_articles_80_quantite} articles représentent␣
↪{proportion_80_quantite:.2f} % du catalogue total.")
```

```
Nombre d'articles représentant 80 % des ventes en quantité : 432
Ces 432 articles représentent 60.59 % du catalogue total.
```

```
C:\Users\t_rak\AppData\Local\Temp\ipykernel_5112\620023092.py:10:
SettingWithCopyWarning:


A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-
docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

C:\Users\t_rak\AppData\Local\Temp\ipykernel_5112\620023092.py:13:
SettingWithCopyWarning:


A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-
docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
```

Etape 5.3 - Analyse des stocks

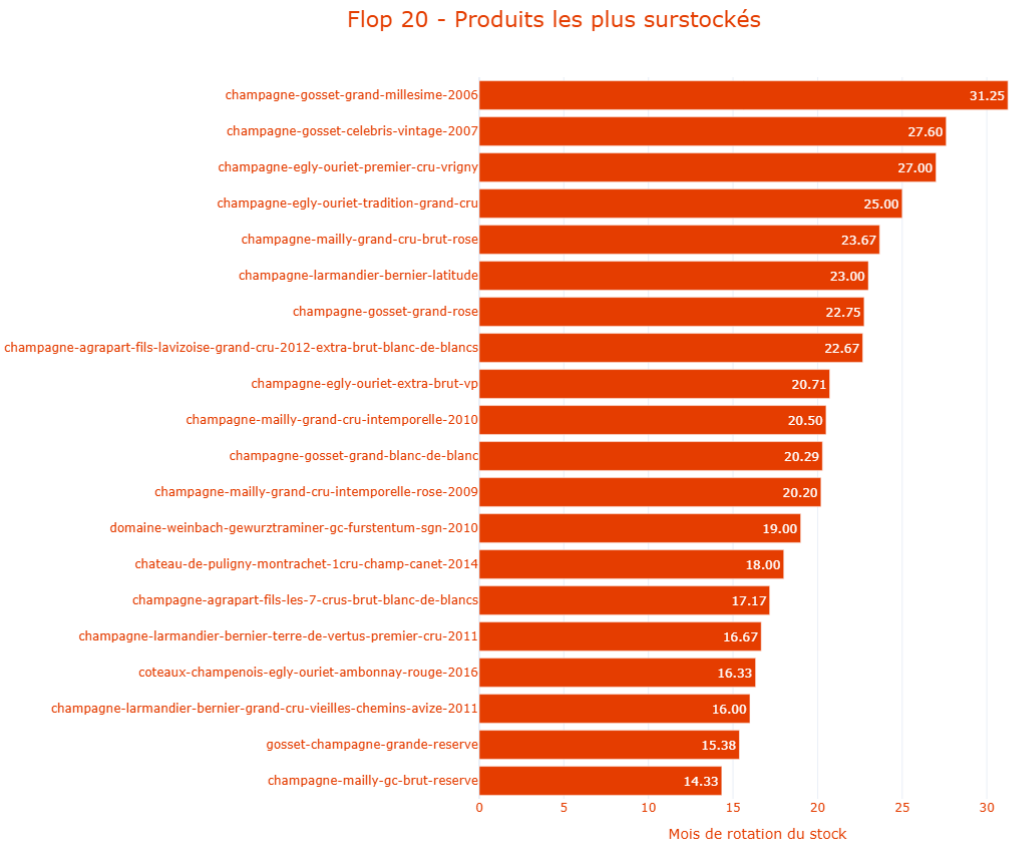
[122]:
```python
df_merge_vf.columns
```

[122]:
```
Index(['id_web', 'product_id', 'onsale_web', 'price', 'stock_quantity',
       'stock_status', 'purchase_price', 'stock_status_2', 'total_sales',
       'tax_status', 'product_type', 'post_name', 'post_modified', 'post_type',
       '_merge', 'z_score_price', 'ca_article', 'part_ca', 'part_ca_cumulee',
       'part_quantite', 'part_quantite_cumulee'],
      dtype='object')
```

[123]:
```
#####################################
# Calculer le nombre de mois de stock #
#####################################
import numpy as np
import seaborn as sns

# -- Calcul de la rotation de stock --
df_merge_vf.loc[:, "rotation_stock"] = df_merge_vf["stock_quantity"] /␣
↪(df_merge_vf["total_sales"])

# -- Nettoyage : remplace les infinis et NaN --
df_merge_vf.loc[:, "rotation_stock"] = df_merge_vf["rotation_stock"].
↪replace([np.inf, -np.inf], 0)
df_merge_vf.loc[:, "rotation_stock"] = df_merge_vf["rotation_stock"].fillna(0)

# -- Tri décroissant --
df_merge_vf = df_merge_vf.sort_values("rotation_stock", ascending=False)

# -- Flop 20 : produits avec le plus de stock (copie explicite) --
# -- Tri décroissant et sélection des 20 articles les plus surstockés --
flop_20 = df_merge_vf.sort_values("rotation_stock", ascending=False).head(20).
↪copy()

# -- Création du graphique avec Plotly Express --
fig = px.bar(
    flop_20,
    x="rotation_stock",
    y="post_name",
    title="Flop 20 - Produits les plus surstockés",
    labels={"post_name": "Article", "rotation_stock": "Mois de rotation du␣
↪stock"},
    text="rotation_stock",
    orientation="h",
    color_discrete_sequence=["#e53d00"]  # couleur uniforme
)

# -- Personnalisation du style --
fig.update_traces(
    texttemplate="%{text:.2f}",
    textposition="inside",
```

```
    marker_color="#e53d00",
    textfont=dict(color="white")
)

fig.update_layout(
    template="plotly_white",
    height=900,
    title_font=dict(color="#e53d00", size=22),
    title_x=0.5,
    xaxis_title="Mois de rotation du stock",
    yaxis_title=None,
    xaxis=dict(title_font=dict(color="#e53d00"),␣
 ↪tickfont=dict(color="#e53d00")),
    yaxis=dict(
        title_font=dict(color="#e53d00"),
        tickfont=dict(color="#e53d00"),
        autorange="reversed"  #  Cette ligne inverse l'ordre visuel : plus␣
 ↪grande barre en haut
    )
)

fig.show()
```

```
C:\Users\t_rak\AppData\Local\Temp\ipykernel_5112\929429111.py:8:
SettingWithCopyWarning:


A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-
docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
```



[124]:
```
#####################################
# Valorisation des stocks en euros #
#####################################

#Création de la colonne Valorisation des stocks en euros
df_merge_vf["valorisation_stock_euros"] = df_merge_vf["stock_quantity"] *␣
↪df_merge_vf["price"]

#Calculer la somme de la colonne "Valorisation_stock_euros"
valeur_totale_stock = df_merge_vf["valorisation_stock_euros"].sum()
print(f"Valeur totale du stock : {valeur_totale_stock:,.2f} €")
```

```
Valeur totale du stock : 494,107.40 €
```

[125]:
```
##############################################
# Valorisation du nombre de produits en stock #
##############################################

#Calculer la somme de la colonne stock quantity
```

```
quantite_totale_stock = df_merge_vf["stock_quantity"].sum()
print(f"Quantité totale du stock : {quantite_totale_stock:,.2f} items")
```

```
Quantité totale du stock : 16,718.00 items
```

Etape 5.4 - Analyse du taux de marge

[127]:
```
############################
# Analyse du taux de marge #
############################

#Création de la colonne Prix HT ##(TVA 20% ??)
df_merge_vf.loc[:, "prix_ht"] = df_merge_vf["price"] / 1.2

#Création de la colonne Taux de marge
df_merge_vf.loc[:, "taux_marge"] = ((df_merge_vf["prix_ht"] -␣
↪df_merge_vf["purchase_price"]) / df_merge_vf["prix_ht"]) * 100
df_merge_vf["taux_marge"] = df_merge_vf["taux_marge"].round(2)

#Afficher le prix minimum de la colonne "taux_marge"
min_marge = df_merge_vf["taux_marge"].min()
print("Taux minimum de la colonne 'taux_marge' :", min_marge)

# Afficher le prix maximum de la colonne "taux_marge"
max_marge = df_merge_vf["taux_marge"].max()
print("Taux maximum de la colonne 'taux_marge' :", max_marge)
```

```
Taux minimum de la colonne 'taux_marge' : -634.99
Taux maximum de la colonne 'taux_marge' : 47.76
```

[128]:
```
#Affichage de la ligne avec un taux de marge inférieur à 0
df_merge_vf[df_merge_vf["taux_marge"] < 0].sort_values("taux_marge").head()
```

[128]:
```
     id_web  product_id  onsale_web  price  stock_quantity stock_status  \
710   12589        4355           1  12.65              97      instock

     purchase_price stock_status_2  total_sales tax_status  … z_score_price  \
710           77.48        instock          0.0    taxable  …     -0.713179

     ca_article part_ca part_ca_cumulee part_quantite  part_quantite_cumulee  \
710        0.0     0.0           100.0           0.0                   100.0

     rotation_stock  valorisation_stock_euros    prix_ht  taux_marge
710             0.0                   1227.05  10.541667     -634.99

[1 rows x 25 columns]
```

[129]:
```
df_merge_vf[df_merge_vf["taux_marge"] < 0].
↪sort_values("taux_marge")["post_name"].head().values
```

[129]: array(['champagne-egly-ouriet-grand-cru-brut-blanc-de-noirs'],
dtype=object)

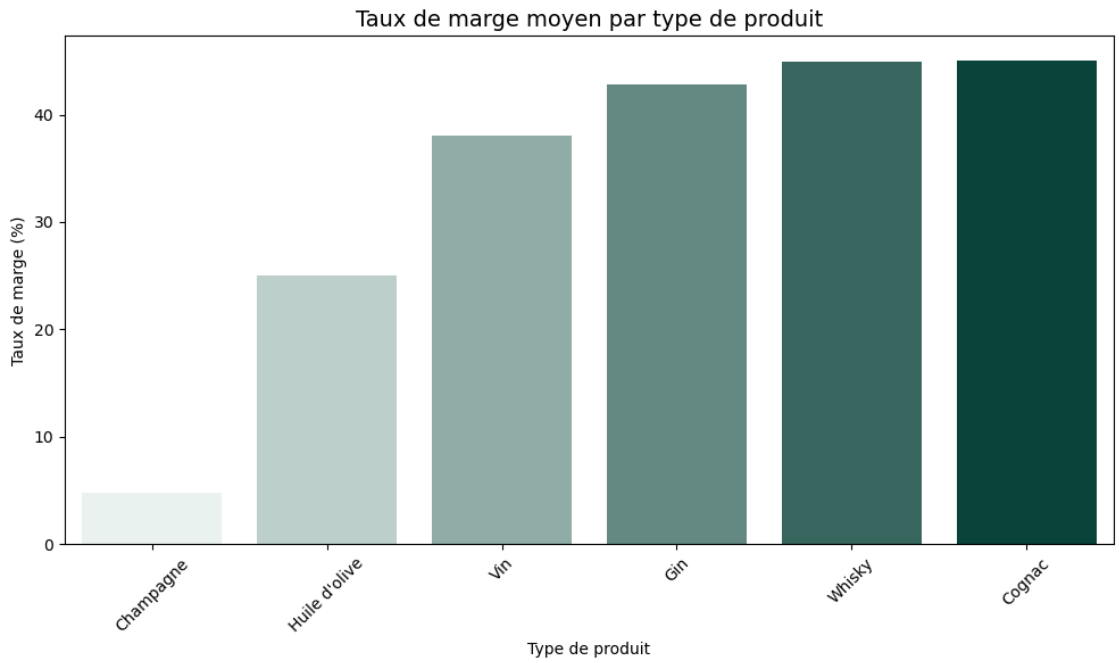
[130]:
```
#Création d'un dataframe avec les taux positifs
df_taux_positif = df_merge_vf[df_merge_vf["taux_marge"] > 0]

#Afficher le prix minimum de la colonne "taux_marge"
min_taux = df_taux_positif["taux_marge"].min()
print("Taux minimum de la colonne 'taux_marge' :", min_taux)

#Afficher le prix maximum de la colonne "taux_marge"
max_taux = df_taux_positif["taux_marge"].max()
print("Taux maximum de la colonne 'taux_marge' :", max_taux)
```

```
Taux minimum de la colonne 'taux_marge' : 22.78
Taux maximum de la colonne 'taux_marge' : 47.76
```

[131]:
```
# Création d'un dataframe avec le taux de marge moyen par type de produit
df_marge_moyenne = df_merge_vf.groupby("product_type")["taux_marge"].mean().
 ↪reset_index()
df_marge_moyenne.columns = ["type_produit", "marge_moyenne"]

# Tri des données pour que les couleurs soient cohérentes avec les valeurs
df_marge_moyenne = df_marge_moyenne.sort_values("marge_moyenne")

# Génère un dégradé de vert foncé au vert clair où les barres les plus hautes␣
 ↪sont plus foncées
import seaborn as sns
import matplotlib.pyplot as plt

palette_verte = sns.light_palette("#004D40", n_colors=len(df_marge_moyenne),␣
 ↪reverse=False)

plt.figure(figsize=(10,6))
sns.barplot(data=df_marge_moyenne, x="type_produit", y="marge_moyenne",␣
 ↪palette=palette_verte)
plt.xticks(rotation=45)
plt.title("Taux de marge moyen par type de produit", fontsize=14)
plt.ylabel("Taux de marge (%)")
plt.xlabel("Type de produit")
plt.tight_layout()
plt.show()
```

```
C:\Users\t_rak\AppData\Local\Temp\ipykernel_5112\4155768843.py:15:
FutureWarning:
```

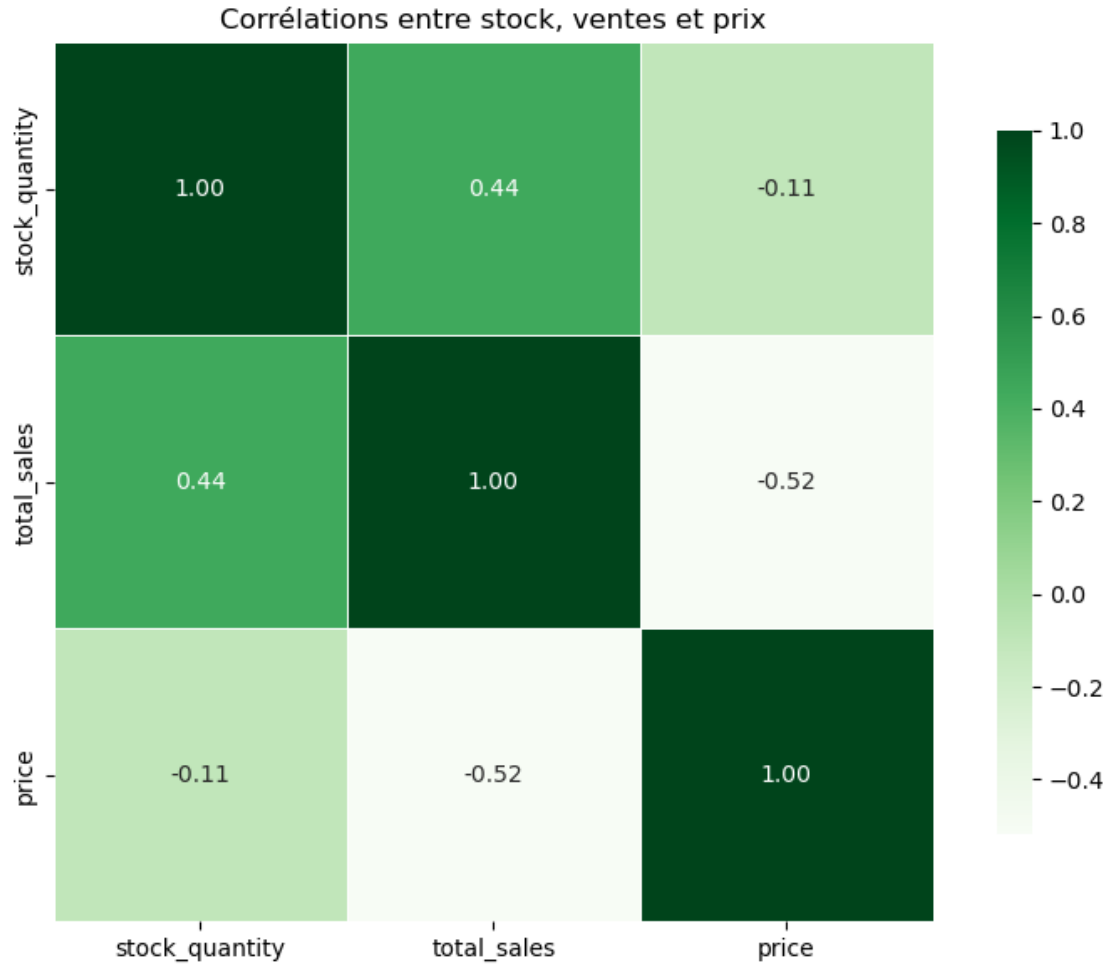
```
Passing `palette` without assigning `hue` is deprecated and will be removed in
v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same
effect.
```



Etape 5.5 - Analyse des corrélations entre les variables stock, sales et price

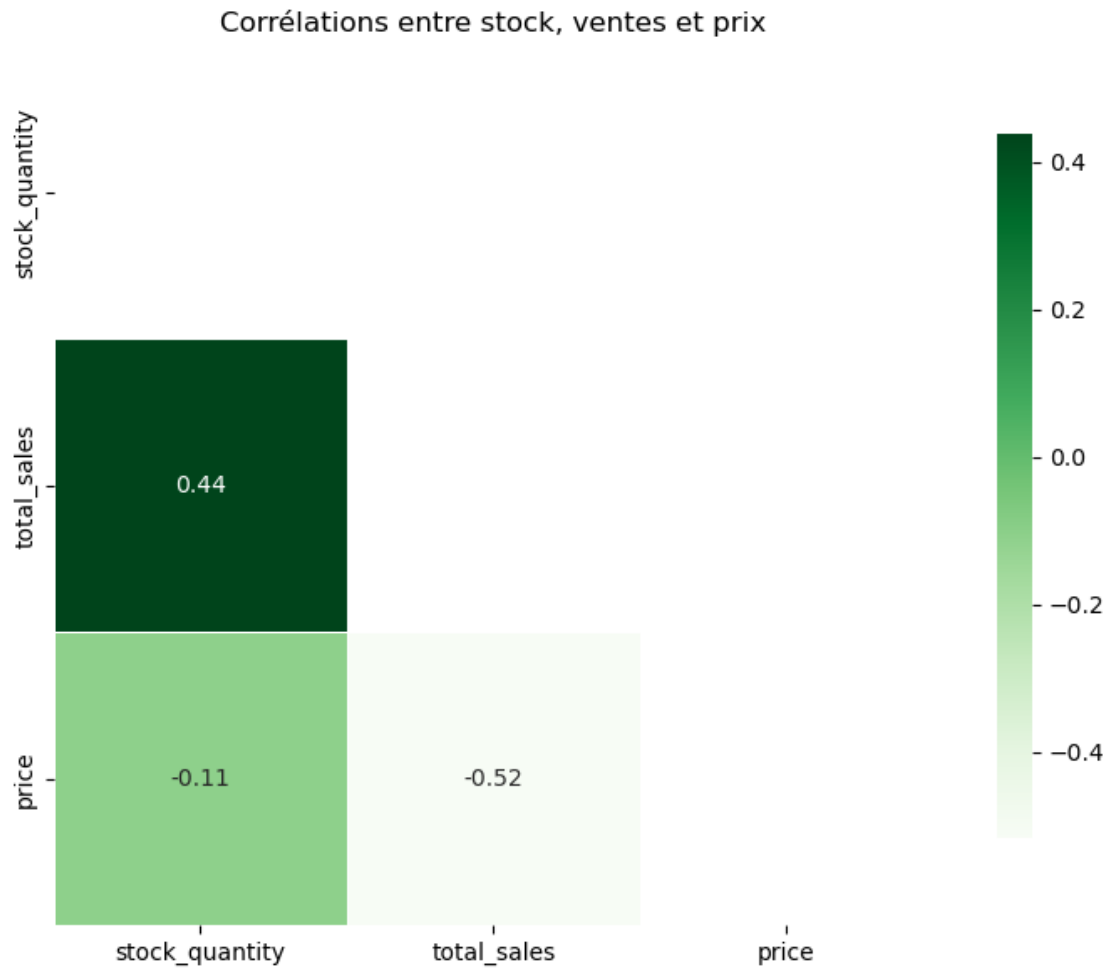
[133]:
```
############################
# Analyse des corrélations #
############################

#Importation de Seaborn

#Création d'une heatmap de corrélation avec les variables stock, sales et price
#1. Variables à garder
variables_corr = df_merge_vf[["stock_quantity", "total_sales", "price"]]

#2. Calcul de la matrice de corrélation
corr_matrix = variables_corr.corr()

#3. Graphique type heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(
    corr_matrix,
    cmap="Greens",  # Tu peux remplacer par "coolwarm", "YlGnBu", etc.
```

```
    annot=True,
    fmt=".2f",
    linewidths=0.5,
    square=True,
    cbar_kws={"shrink": 0.8}
)
plt.title("Corrélations entre stock, ventes et prix")
plt.tight_layout()
plt.show()
```



[134]:
```
#On peut également créer un mask pour n'afficher qu'une demi heatmap
mask = np.triu(np.ones_like(corr_matrix, dtype=bool))  # Cache la diagonale et␣
 ↪la partie supérieure

plt.figure(figsize=(8, 6))
sns.heatmap(
```

```
    corr_matrix,
    mask=mask,
    cmap="Greens",  # Tu peux remplacer par "coolwarm", "YlGnBu", etc.
    annot=True,
    fmt=".2f",
    linewidths=0.5,
    square=True,
    cbar_kws={"shrink": 0.8}
)
plt.title("Corrélations entre stock, ventes et prix")
plt.tight_layout()
plt.show()
```



[135]:
```
#Que peut-on conclure des corrélations ?
print("Les produits bon marché se vendent mieux :\nCorrélation négative entre␣
↪prix et ventes  -0.52\n- Plus le prix est bas, plus les ventes augmentent.")
```

```
print("\nLes produits très vendus ont un stock élevé :\nCorrélation positive␣
↪entre stock et ventes  +0.44\n- La logistique suit bien la demande.")
print("\nLe prix influence peu le stock :\nCorrélation très faible entre prix␣
↪et stock  -0.11\n- D'autres variables influencent l'approvisionnement.")
```

```
Les produits bon marché se vendent mieux :
Corrélation négative entre prix et ventes  -0.52
- Plus le prix est bas, plus les ventes augmentent.

Les produits très vendus ont un stock élevé :
Corrélation positive entre stock et ventes  +0.44
- La logistique suit bien la demande.

Le prix influence peu le stock :
Corrélation très faible entre prix et stock  -0.11
- D'autres variables influencent l'approvisionnement.
```

Etape 5.6 - Mise à disposition de la nouvelle table sur un fichier Excel

[137]:
```
!pip install openpyxl
```

```
Requirement already satisfied: openpyxl in c:\users\t_rak\anaconda3\lib\site-
packages (3.1.5)
Requirement already satisfied: et-xmlfile in c:\users\t_rak\anaconda3\lib\site-
packages (from openpyxl) (1.1.0)
```

[138]:
```
#Mettre le dataset df_merge sur un fichier Excel
df_merge_vf.to_excel("df_merge_export.xlsx", index=False)

#Cette étape peut être utile pour partager le résultat du dataset obtenu avec␣
↪les équipes.
```